

Topology

- Bus (simplest)
- Point-to-point connections (most costly)
- Crossbar
- Ring
- Tree
- Omega
- Hypercube
- Mesh
- Torus
- Butterfly
- ...

Properties of a Topology/Network

- **Bisection Bandwidth**

- Often used to describe network performance
- **Cut network in half and sum bandwidth of links severed**
 - (Min # channels spanning two halves) * (BW of each channel)
- Meaningful only for recursive topologies
- Can be misleading, because does not account for switch and routing efficiency (and certainly not execution time)

- **Blocking vs. Non-Blocking**

- If connecting any permutation of sources & destinations is possible, network is non-blocking; otherwise network is blocking.
- Rearrangeable non-blocking: Same as non-blocking but might require rearranging connections when switching from one permutation to another.

Metrics to Evaluate Interconnect Topology

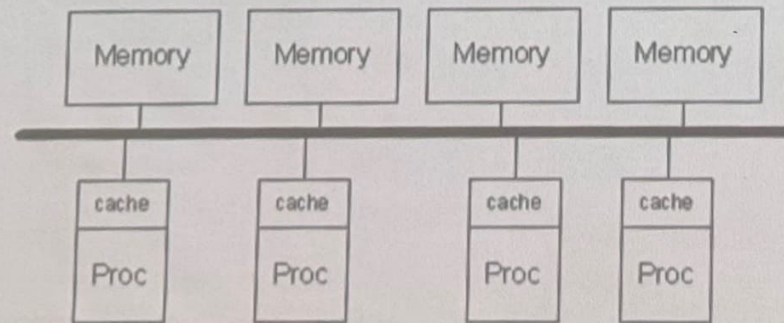
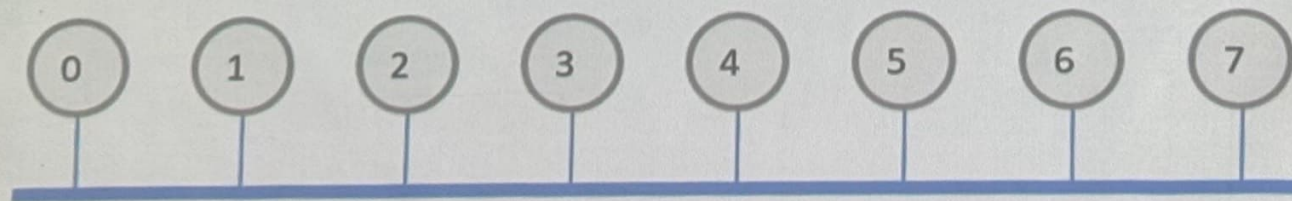
- Cost
- Latency (in hops, in nanoseconds)
- Contention
- Many others exist you should think about
 - Energy
 - Bandwidth
 - Overall system performance



Bus

All nodes connected to a single link

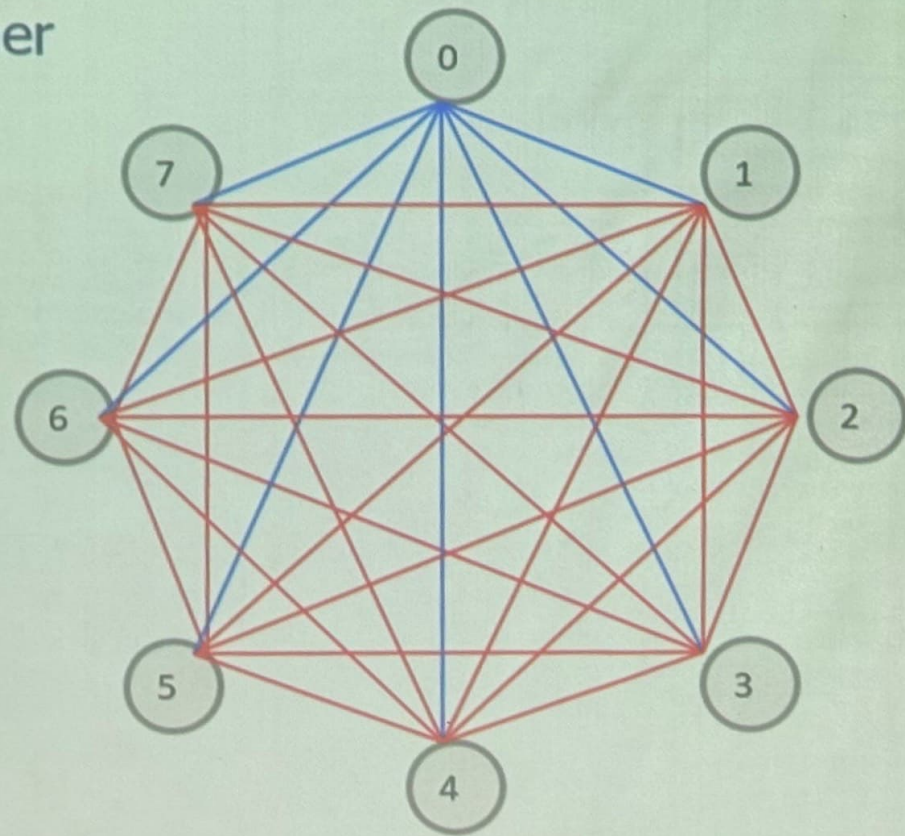
- + Simple + Cost effective for a small number of nodes
- + Easy to implement coherence (snooping and serialization)
- Not scalable to large number of nodes (limited bandwidth, electrical loading → reduced frequency)
- High contention → fast saturation



Point-to-Point

Every node connected to every other
with direct/isolated links

- + Lowest contention
- + Potentially lowest latency
- + Ideal, if cost is no issue
- Highest cost
 - $O(N)$ connections/ports per node
 - $O(N^2)$ links
- Not scalable
- How to lay out on chip?



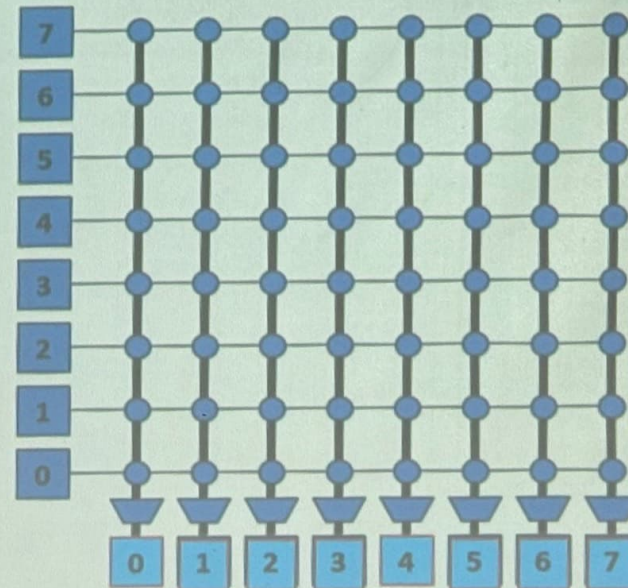
Crossbar

- Every node connected to every other with a shared link for each destination
- Enables concurrent transfers to non-conflicting destinations
- Could be cost-effective for small number of nodes

- + Low latency and high throughput
- Expensive
- Not scalable $\rightarrow O(N^2)$ cost
- Difficult to arbitrate as N increases

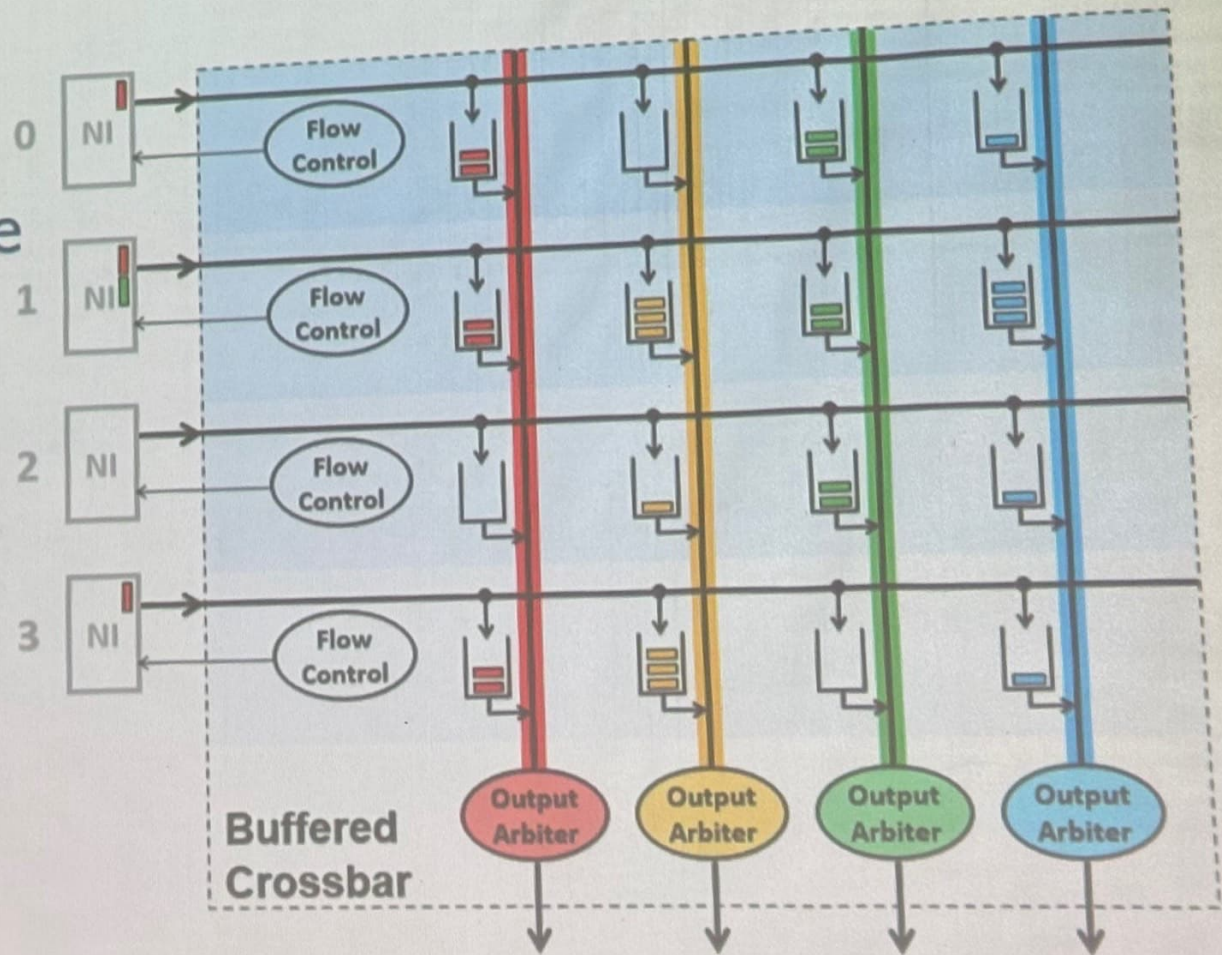
Used in core-to-cache-bank networks in

- IBM POWER5
- Sun Niagara I/II



Bufferless and Buffered Crossbars

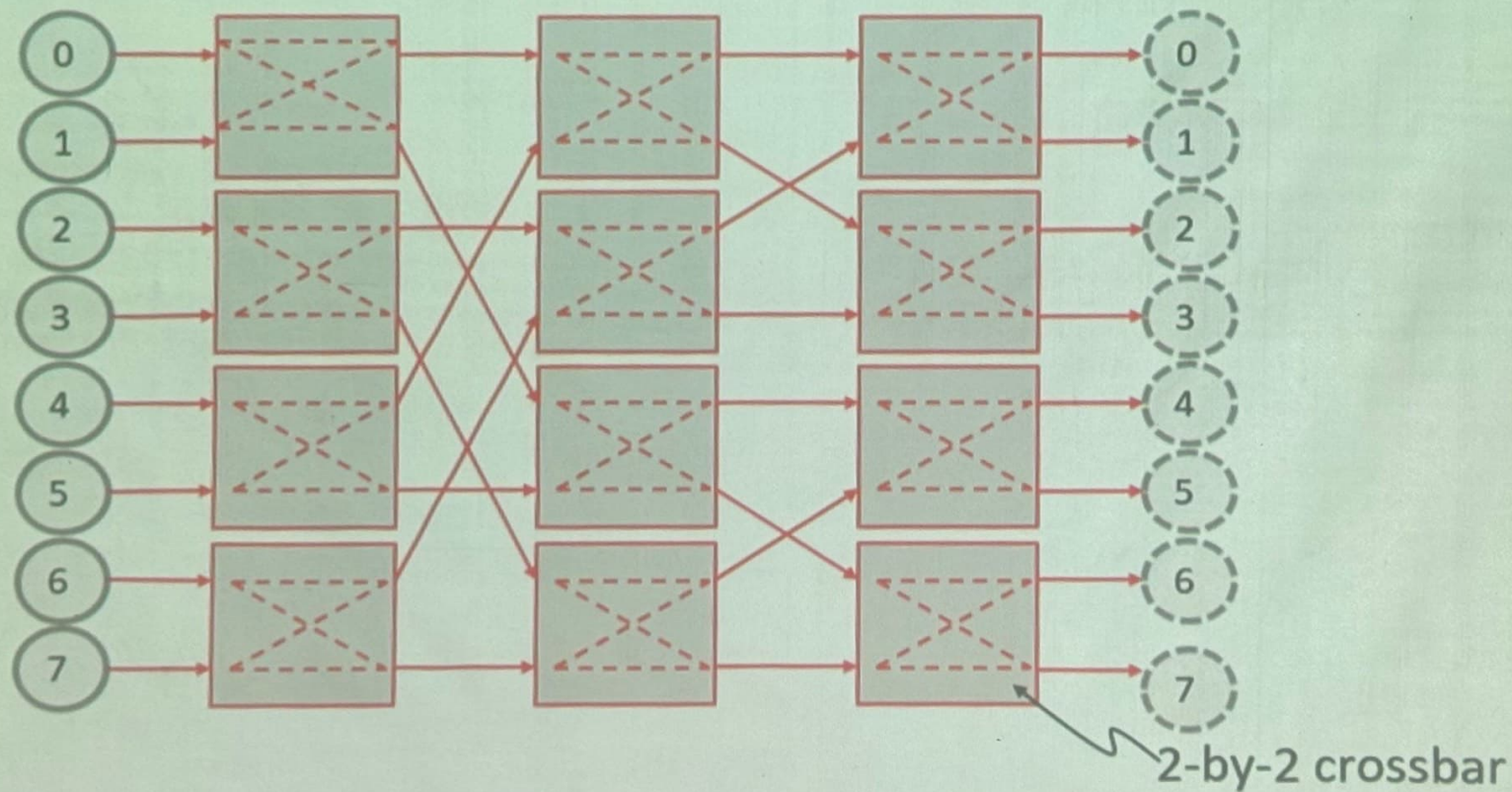
- + Simpler arbitration/scheduling
- + Efficient support for variable-size packets
- Requires N^2 buffers



Can We Get Lower Cost than A Crossbar?

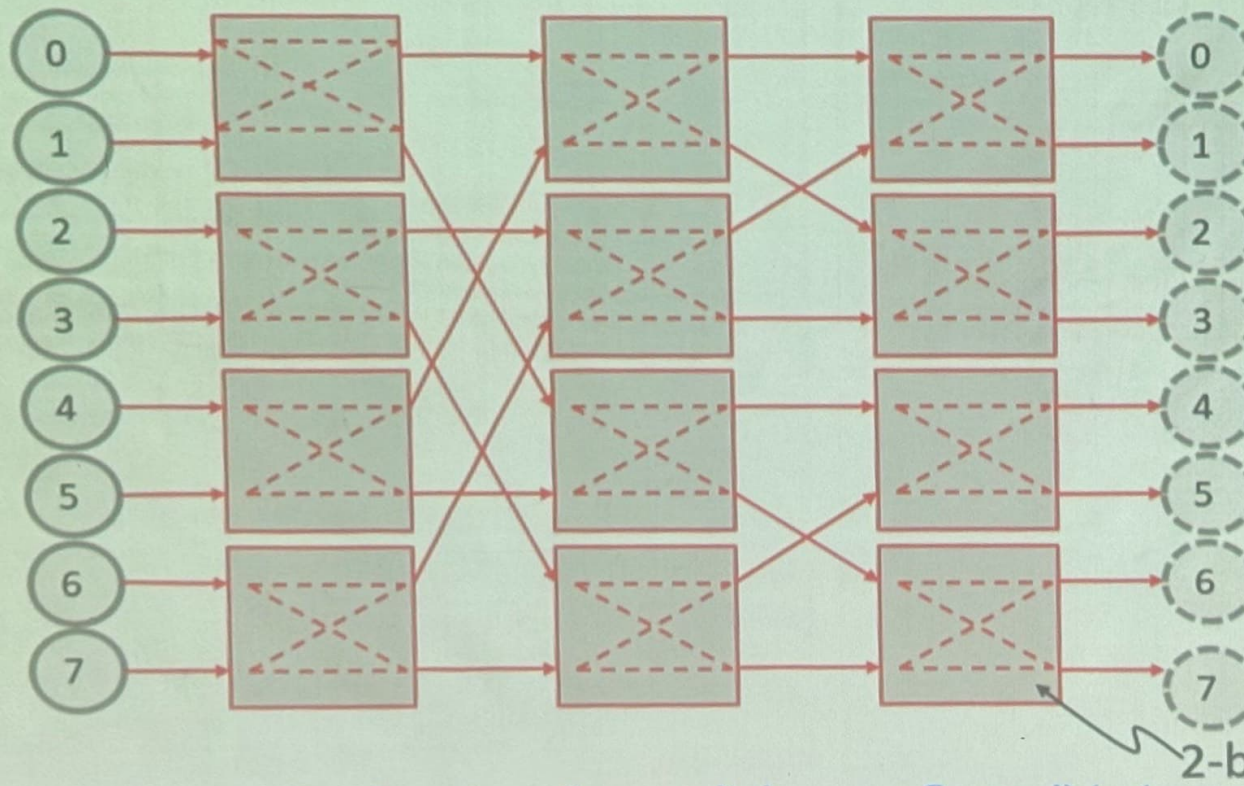
- Yet still have low contention compared to a bus?
- Idea: Multistage networks

Multistage Networks



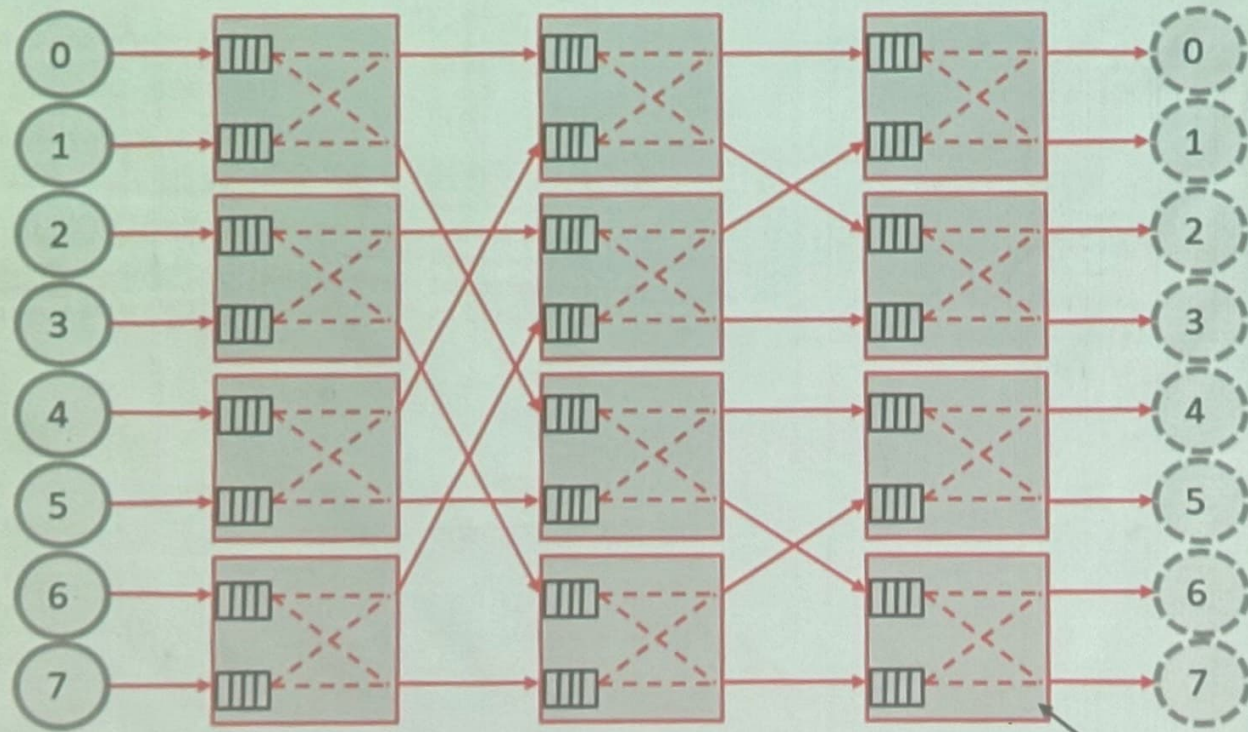
- A multistage network **restricts concurrent Tx-Rx pairs vs. a crossbar**
- But, it is **less costly than crossbar**, e.g., $O(N \log N)$ for Butterfly

Multistage Networks (Circuit Switched)



- Source-destination path completely set up (i.e., switches configured) to transmit data before data is transmitted – pre-configure switches
- No need for buffering since switching is static once path is set up

Multistage Networks (Packet Switched)



- Packets "hop" from router to router, pending availability of the next required switch and buffer

Circuit vs. Packet Switching

- **Circuit switching** sets up full path before transmission
 - Establish route then send data
 - No one else can use those links while "circuit" is set
 - + faster arbitration
 - + no buffering
 - setting up and bringing down "path" takes time
 - path cannot be used by multiple flows concurrently
- **Packet switching** performs routing per packet in each router
 - Route each packet individually (possibly via different paths)
 - If link is free, any packet can use it
 - potentially slower --- must dynamically switch
 - need handling contention (e.g., via buffering)
 - + no setup, bring down time
 - + more flexible, does not underutilize links

Switching vs. Topology

- Circuit/packet switching choice independent of topology
- It is a higher-level protocol on how a message gets sent to a destination
- However, some topologies are more amenable to circuit vs. packet switching

Heterogeneous Interconnects (in Tiler)

- Topology: 2D Mesh
- Five networks
- Four packet switched
 - Dimension order routing, wormhole flow control
 - TDN: Cache request packets
 - MDN: Response packets
 - IDN: I/O packets
 - UDN: Core to core messaging
- One circuit switched
 - STN: Low-latency, high-bandwidth static network
 - Streaming data

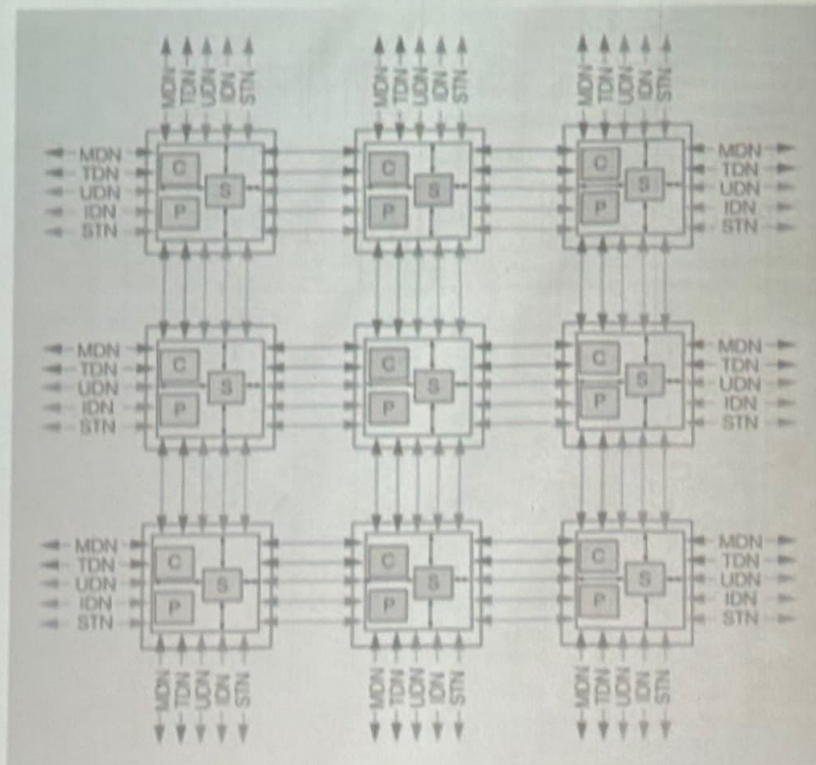
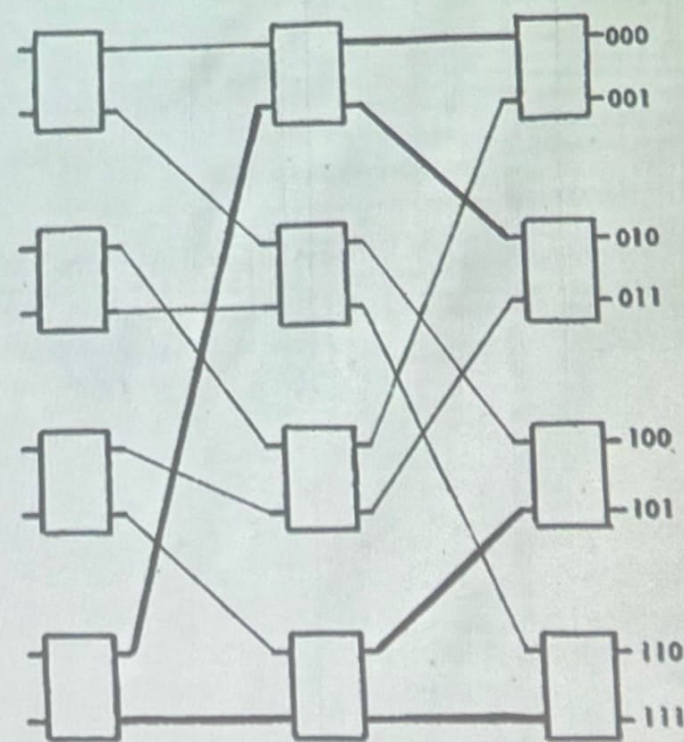


Figure 3. A 3×3 array of tiles connected by networks. (MDN: memory dynamic network; TDN: tile dynamic network; UDN: user dynamic network; IDN: I/O dynamic network; STN: static network.)

Another Multistage Network: Delta Network

- Single path from source to destination
- Each stage has different routers
- Proposed to replace costly crossbars as processor-memory interconnect
- Janak H. Patel, "Processor-Memory Interconnections for Multiprocessors," ISCA 1979.



8x8 Delta network

Another Multistage Network: Omega Network

- Single path from source to destination
- All stages are the same
- Used in NYU Ultracomputer
- Gottlieb et al. "The NYU Ultracomputer - Designing an MIMD Shared Memory Parallel Computer," IEEE Trans. On Comp., 1983.

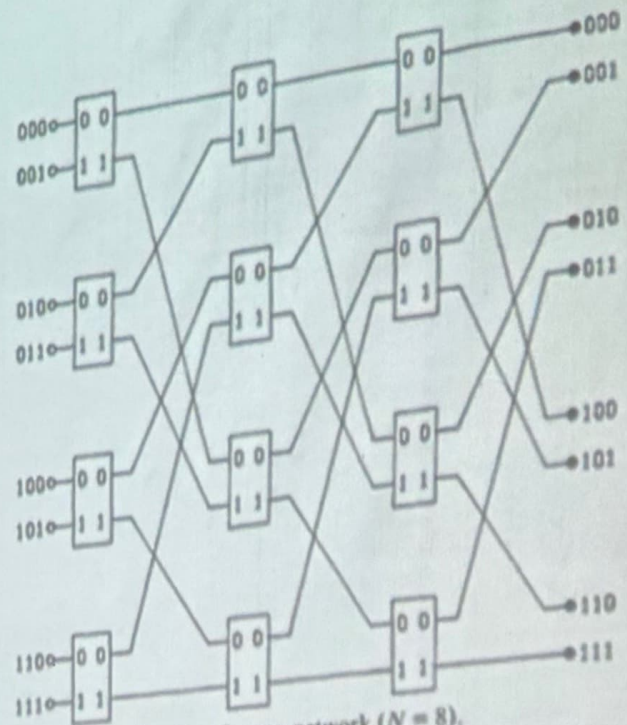
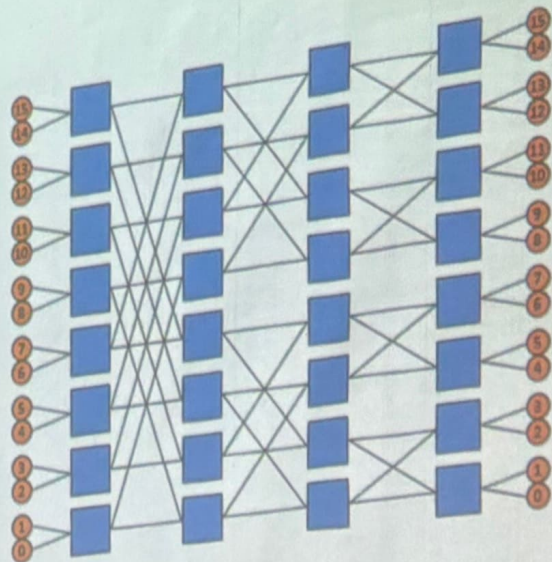


Fig. 2. Omega-network ($N=8$).

Butterfly

- Equivalent to Omega Network
- Indirect
- Used in BBN Butterfly
- Conflicts can cause "*tree saturation*"
 - Randomization of route selection helps

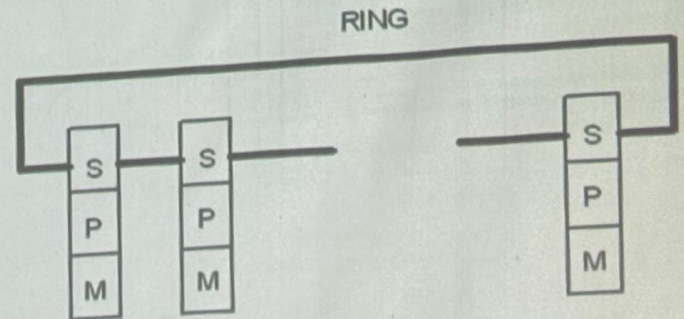


Ring

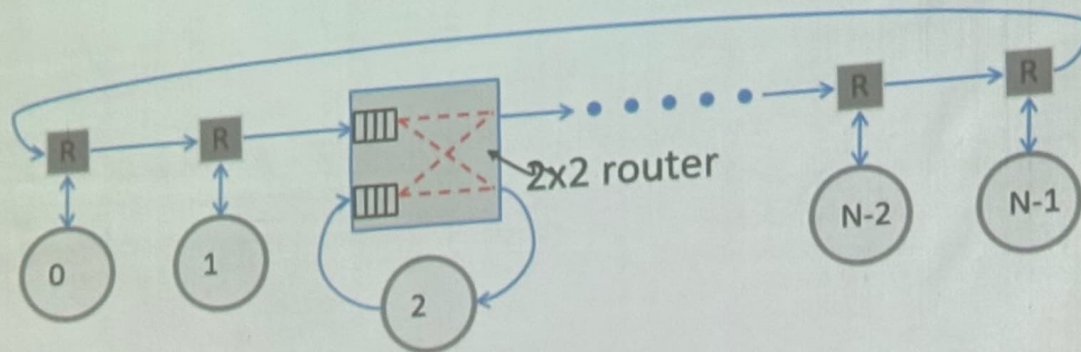
Each node connected to exactly two other nodes. Nodes form a continuous pathway such that packets can reach any node.

- + Cheap: $O(N)$ cost
- High latency: $O(N)$
- Not easy to scale
 - Bisection bandwidth remains constant

Used in Intel Haswell,
Intel Larrabee, IBM Cell,
many commercial systems today



Unidirectional Ring



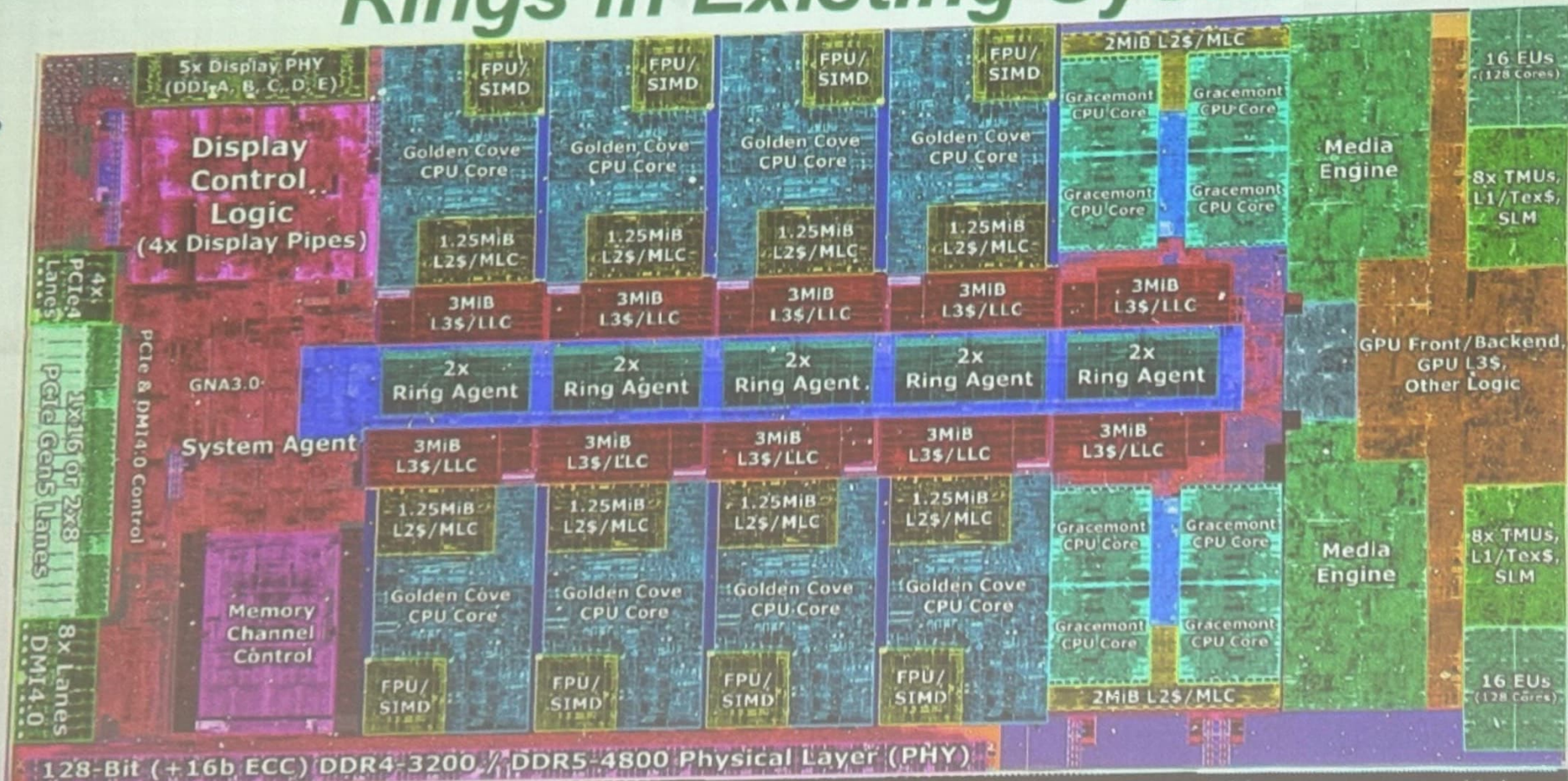
- Single directional pathway
- Simple topology and implementation
 - Reasonable performance if N and performance needs (bandwidth & latency) still moderately low
 - $O(N)$ cost
 - $N/2$ average hops; latency depends on utilization

Bidirectional Rings

Multi-directional pathways, or multiple rings

- + Reduces latency
- + Improves scalability
- Slightly more complex injection policy (need to select which ring to inject a packet into)

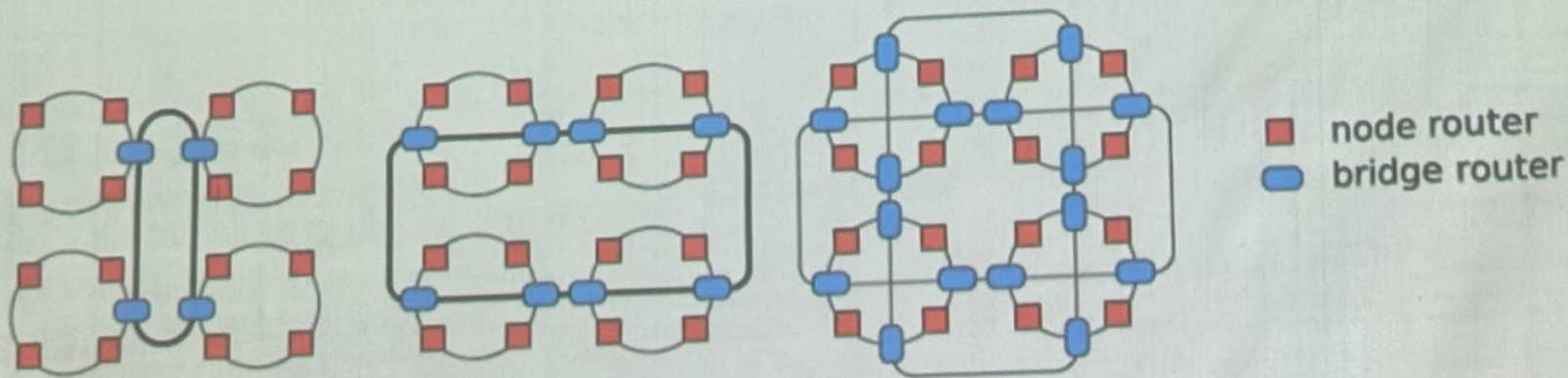
Rings in Existing Systems



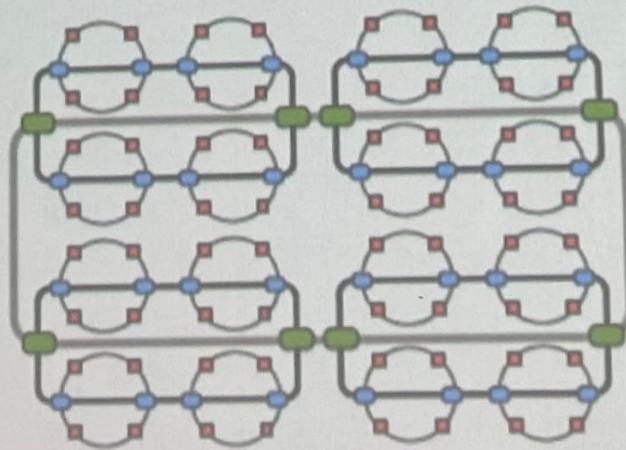
10nm ESF=Intel 7 Alder Lake die shot (~209mm²) from Intel: <https://www.intel.com/content/www/us/en/newsroom/news/12th-gen-core-processors.html>
Die shot interpretation by Locuza, October 2021

Intel Alder Lake, 2021

Hierarchical Rings



(a) 4-, 8-, and 16-bridge hierarchical ring topologies.



(b) Three-level hierarchy (8x8).

- + More scalable
- + Lower latency
- More complex

Mesh

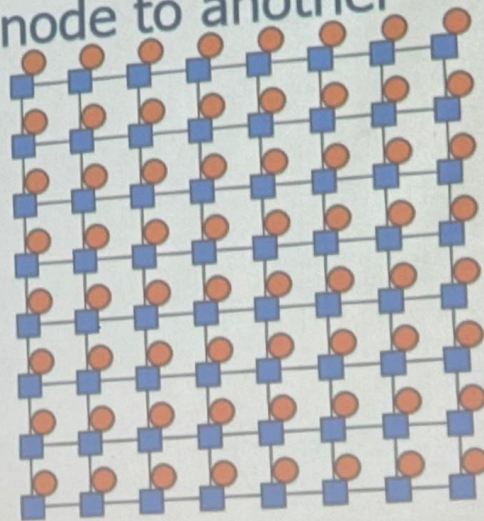
- Each node connected to 4 neighbors (N, E, S, W)

- $O(N)$ cost

- Average latency: $O(\sqrt{N})$

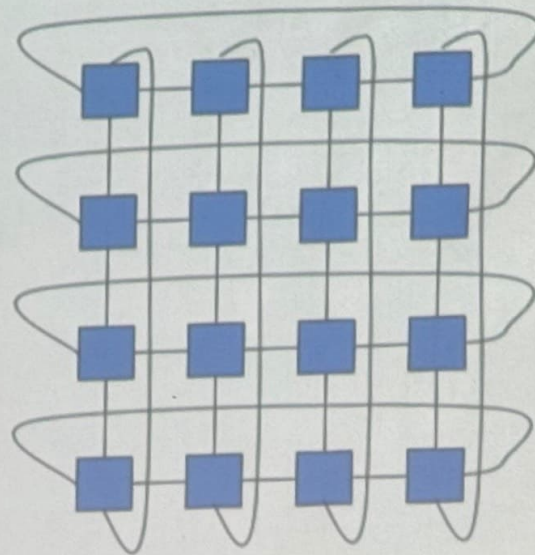
- Easy to layout on-chip: regular and equal-length links

- Path diversity: many ways to get from one node to another

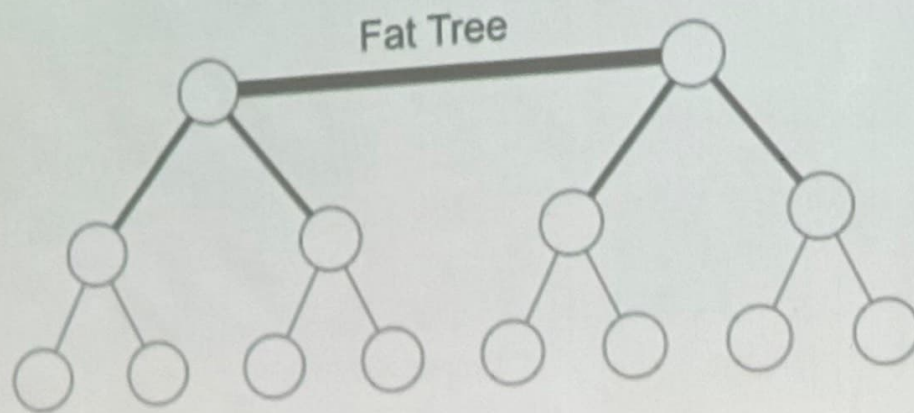
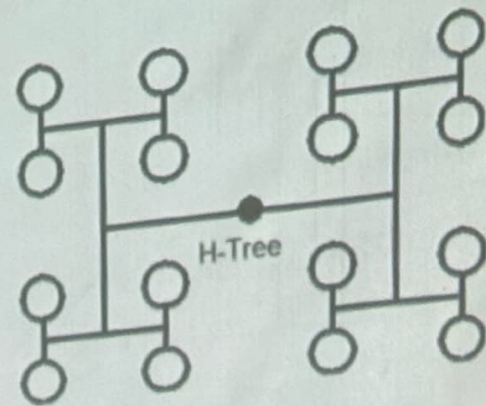


Torus

- Mesh is not symmetric on edges: performance very sensitive to placement of task on edge vs. middle
- Torus avoids this problem
- + Higher path diversity (and bisection bandwidth) than mesh
- Higher cost
- Harder to lay out on-chip
- Unequal link lengths

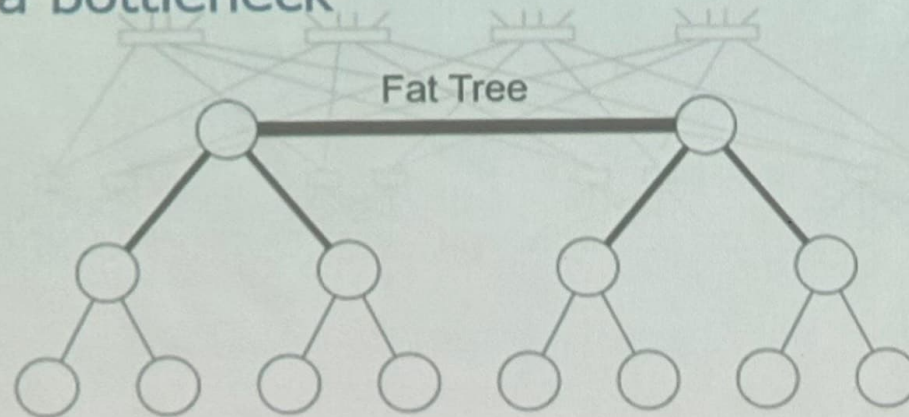
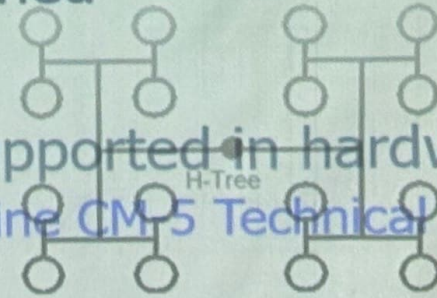


Planar, hierarchical topology
Latency: $O(\log N)$
Good for local traffic



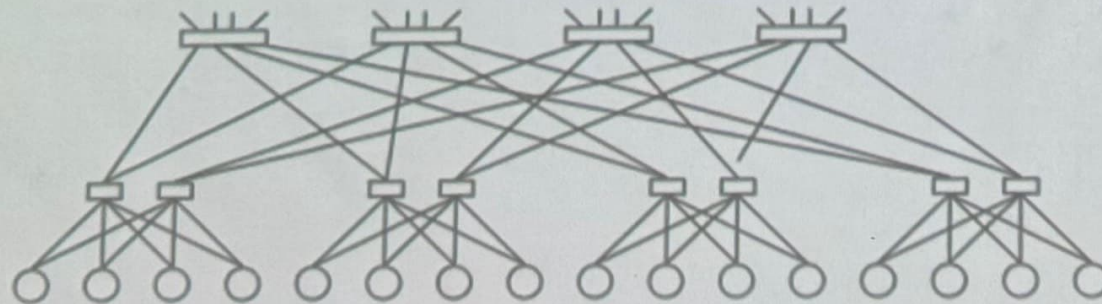
CM-5 Fat Tree

- Fat tree based on 4x2 switches, packet switched
 - Randomized routing on the way up
 - Combining multicast, reduction operators supported in hardware
 - + Cheap: $O(N)$ cost
 - + Easy to Layout
 - Root can become a bottleneck
- Fat trees avoid



CM-5 Fat Tree

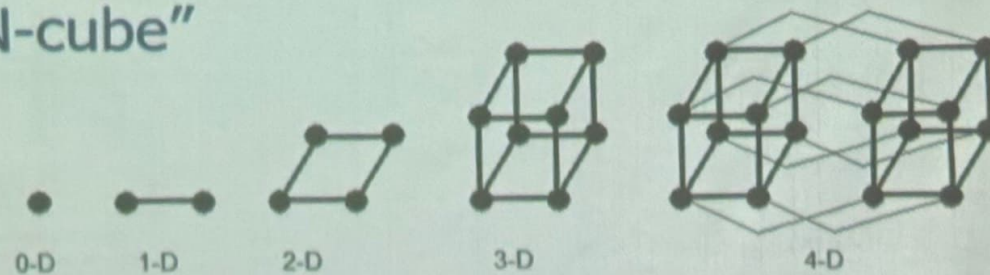
- Fat tree based on 4x2 switches, packet switched
- Randomized routing on the way up
- Combining, multicast, reduction operators supported in hardware
 - Thinking Machines Corp., "The Connection Machine CM-5 Technical Summary," Jan. 1992.



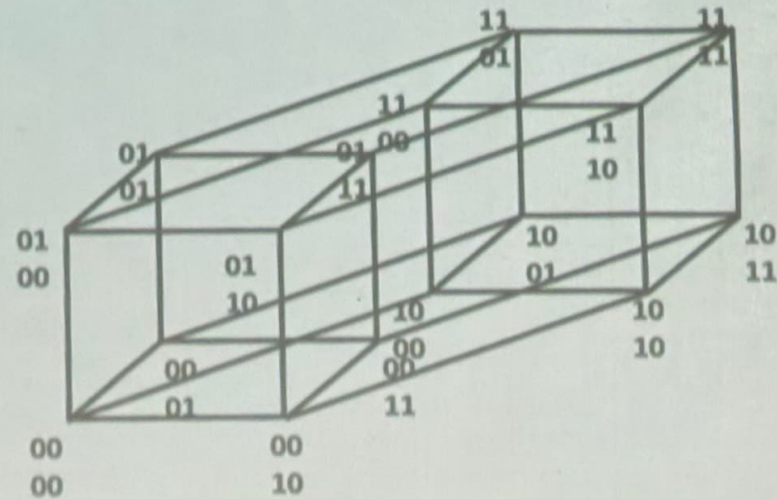
CM-5 Thinned Fat Tree

Hypercube

- "N-dimensional cube" or "N-cube"



- Latency: $O(\log N)$
- Radix: $O(\log N)$
- #links: $O(N \log N)$



Routing Mechanism

• Arithmetic

- Simple arithmetic to determine route in regular topologies
- Dimension order routing in meshes/tori

• Source Based

- Source specifies output port for each switch in route
- + Simple switches
 - no control state: strip output port off header
- Large header

• Table Lookup Based

- Index into table for output port
- + Small header
- More complex switches

Routing Algorithm

- Three Types
 - **Deterministic:** always chooses the same path for a communicating source-destination pair
 - **Oblivious:** chooses different paths, without considering network state
 - **Adaptive:** can choose different paths, adapting to the state of the network
- How to adapt
 - Local/global feedback

Deterministic Routing

- All packets between the same (source, dest) pair take the same path

- **Dimension-order routing**

- First traverse dimension X, then traverse dimension Y
- E.g., XY routing (used in Cray T3D, and many on-chip networks)

+ Simple

+ Deadlock freedom (no cycles in resource allocation)

Deadlock

- No forward progress
- Caused by circular dependencies on resources
- Each packet waits for a buffer occupied by another packet downstream



Handling Deadlock

- Avoid cycles in routing
 - Dimension order routing
 - Cannot build a circular dependency
 - Restrict the “turns” each packet can take
- Avoid deadlock by adding more buffering (escape paths)
- Detect and break deadlock
 - Preemption of buffers

Turn Model to Avoid Deadlock

• Idea

- Analyze directions in which packets can turn in the network
- Determine the cycles that such turns can form
- Prohibit just enough turns to break possible cycles

- Glass and Ni, "The Turn Model for Adaptive Routing," ISCA 1992.

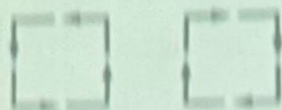


FIG. 3. The possible turns and simple cycles in a two-dimensional mesh.



FIG. 3. The four turns allowed by the xy routing algorithm.

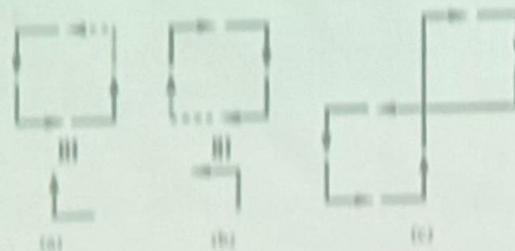


FIG. 4. Six turns that complete the cycles and allow deadlock.

Adaptive Routing

- Minimal adaptive

- Router uses network state (e.g., downstream buffer occupancy) to pick which “productive” output port to send a packet to
- Productive output port: port that gets the packet closer to its destination
- + Aware of local congestion
- Minimality restricts achievable link utilization (load balance)

- Non-minimal (fully) adaptive

- “Misroute” packets to non-productive output ports based on network state
- + Can achieve better network utilization and load balance
- Need to guarantee livelock freedom

More on Adaptive Routing

- Can avoid faulty links/routers
- Idea: **Route around faults**
- + Deterministic routing cannot handle faulty components
 - Need to change the routing table to disable faulty routes
 - Assuming the faulty link/router is detected

One relatively recent example:

Fattah et al., "A Low-Overhead, Fully-Distributed, Guaranteed Delivery Routing Algorithm for Faulty Network-on-Chip"
2015.

Recall: Circuit vs. Packet Switching

- **Circuit switching** sets up full path before transmission
 - Establish route then send data
 - No one else can use those links while "circuit" is set
 - + faster arbitration
 - + no buffering
 - setting up and bringing down "path" takes time
 - path cannot be used by multiple flows concurrently
- **Packet switching** performs routing per packet in each router
 - Route each packet individually (possibly via different paths)
 - If link is free, any packet can use it
 - potentially slower --- must dynamically switch
 - need handling contention (e.g., via buffering)
 - + no setup, bring down time
 - + more flexible, does not underutilize links

Recall: Heterogeneous Interconnects (in Tiler)

- Topology: 2D Mesh
- Five networks
- Four packet switched
 - Dimension order routing, wormhole flow control
 - TDN: Cache request packets
 - MDN: Response packets
 - IDN: I/O packets
 - UDN: Core to core messaging
- One circuit switched
 - STN: Low-latency, high-bandwidth static network
 - Streaming data

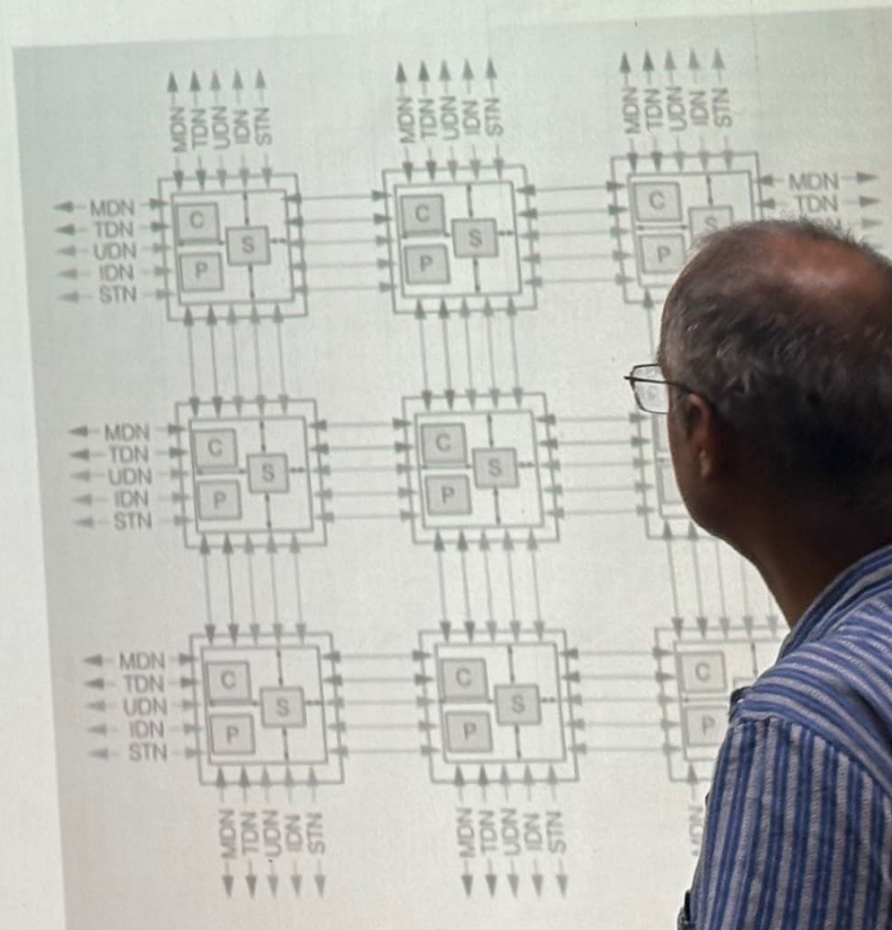
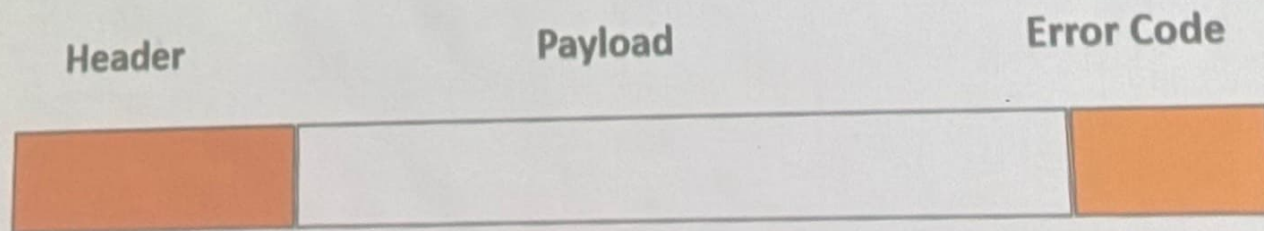


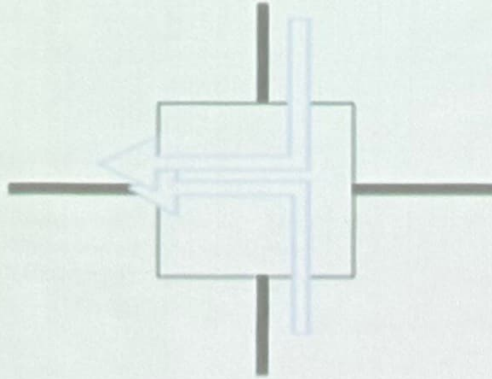
Figure 3. A 3×3 array of tiles connected by networks. (MDN: tile dynamic network; UDN: user dynamic network; IDN: static network.)

Packet Switched Networks: Packet Format

- Header
 - routing and control information
- Payload
 - carries data (non HW specific information)
 - can be further divided (framing, protocol stacks...)
- Error Code
 - generally at tail of packet so it can be generated on the way out



Handling Contention



- Two packets trying to use the same link at the same time
- What do you do?
 - Buffer one
 - Drop one
 - Misroute one (deflection)
- Tradeoffs?

Flow Control Methods

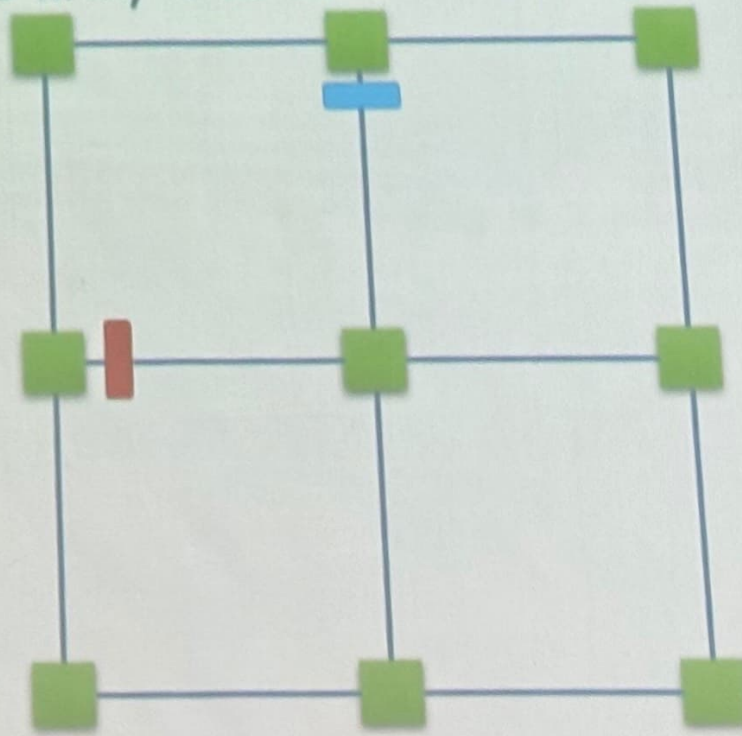
- Circuit switching
- Bufferless (Packet/flit based)
- Store and forward (Packet based)
- Virtual cut through (Packet based)
- Wormhole (Flit based)

Circuit Switching Revisited

- Resource allocation granularity is large
- Idea: Pre-allocate resources across multiple switches for a given “flow”
- Need to send a probe to set up the path for pre-allocation
- + No need for buffering
- + No contention (flow’s performance is isolated)
- + Can handle arbitrary message sizes
- Lower link utilization: two flows cannot use the same link
- Handshake overhead to set up a “circuit”

Bufferless Deflection Routing

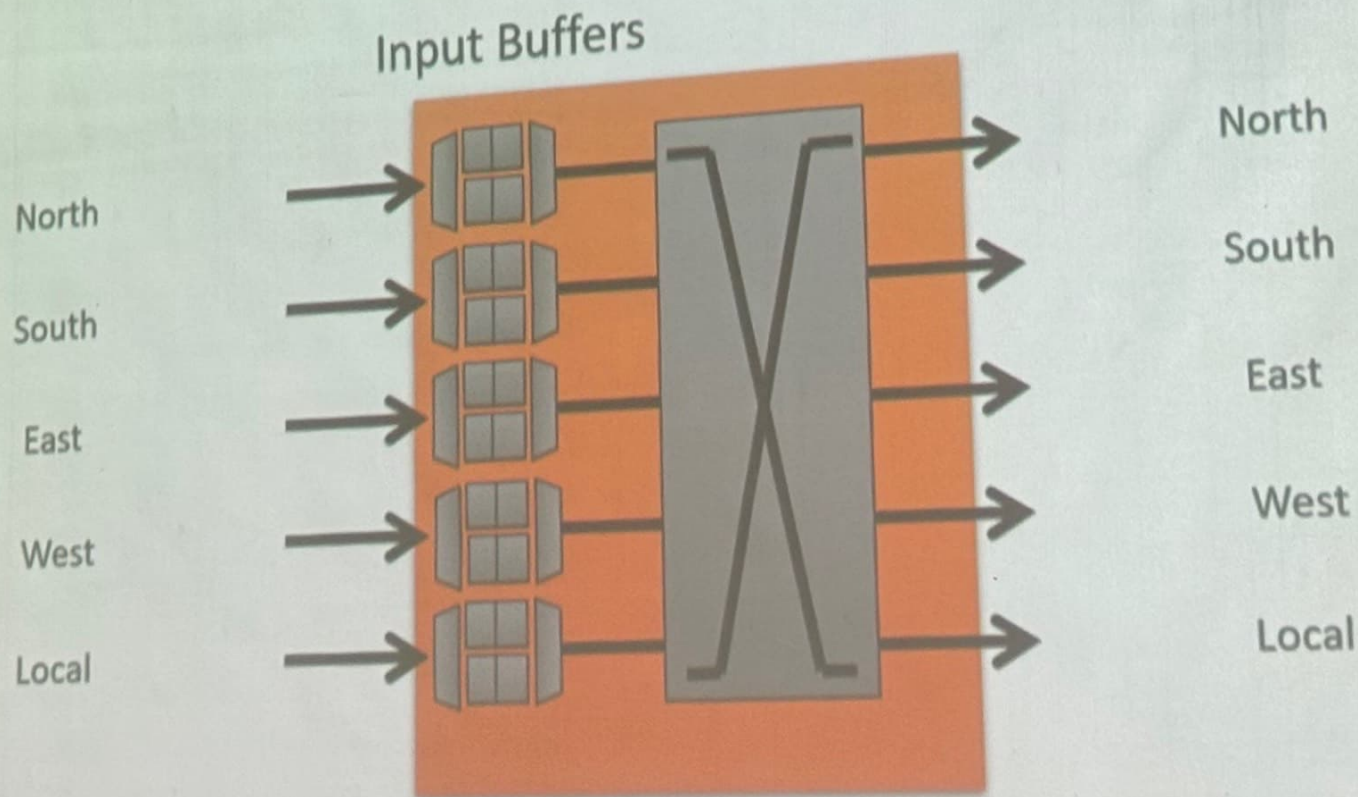
- **Key idea:** Packets are never buffered in the network. When two packets contend for the same link, one is **deflected**.¹



¹Baran, "On Distributed Communication Networks." RAND Tech. Report., 1962 / IEEE Trans.Comm., 1964.

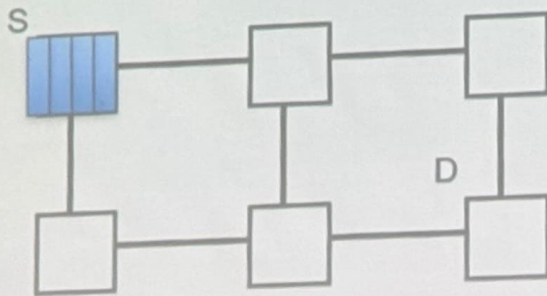
Bufferless Deflection Routing

- Input buffers are eliminated: packets are buffered in **pipeline latches** and on **network links**



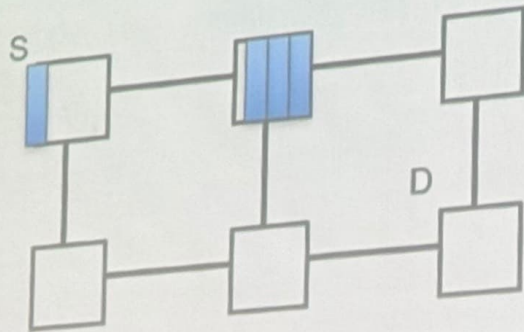
Store and Forward Flow Control

- Packet-based flow control
- Store and Forward
 - Packet copied entirely into network router before moving to the next node
 - Flow control unit is the entire packet



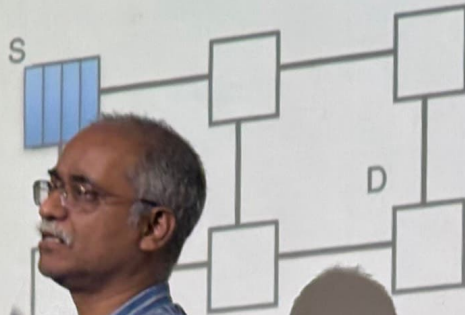
Store and Forward Flow Control

- Packet-based flow control
- Store and Forward
 - Packet copied entirely into network router before moving to the next node
 - Flow control unit is the entire packet
- Leads to high per-packet latency
- Requires buffering for entire packet in each node



Cut-Through Flow Control

- Another form of packet-based flow control
- Start forwarding as soon as header is received and resources (buffer, channel, etc) allocated



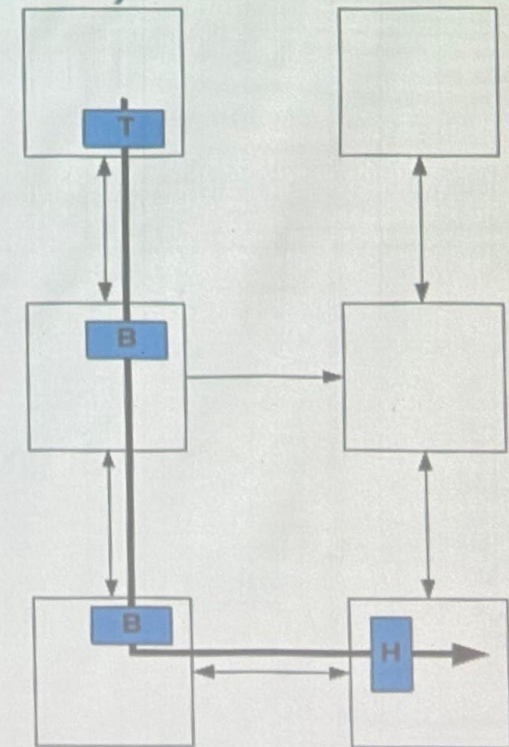
Cut-Through Flow Control

- What to do if output port is blocked?
- Lets the tail continue when the head is blocked, absorbing the whole message into a single switch.
 - Requires a buffer large enough to hold the largest packet.
- Degenerates to store-and-forward with high contention
- **Can we do better?**



Wormhole Flow Control

- Packets broken into (potentially) smaller flits (buffer/bw allocation unit)
- Flits are sent across the fabric in a *wormhole fashion*
 - Body follows head, tail follows body
 - Pipelined
 - If head blocked, rest of packet stops
 - Routing (src/dest) information only in head



Wormhole Flow Control

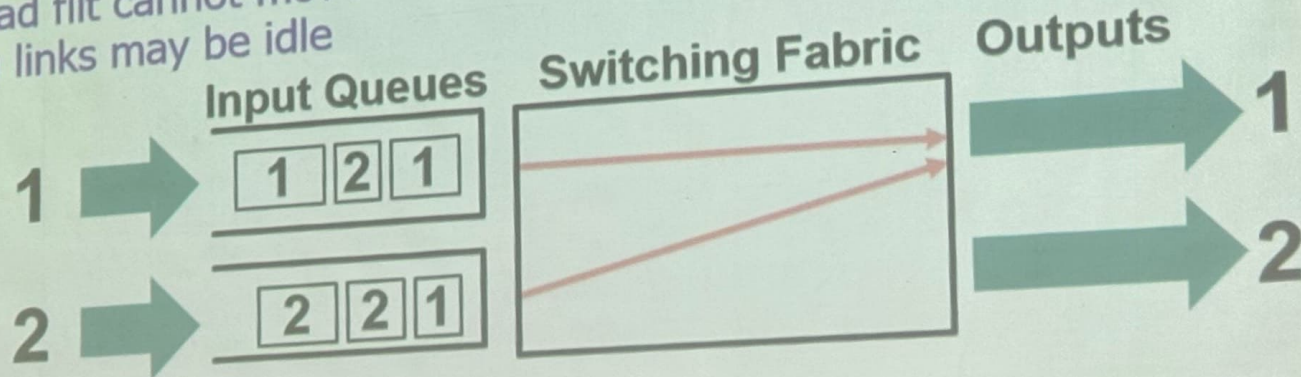
- Advantages over “store and forward” flow control

- + Lower latency
- + More efficient buffer utilization

- Limitations

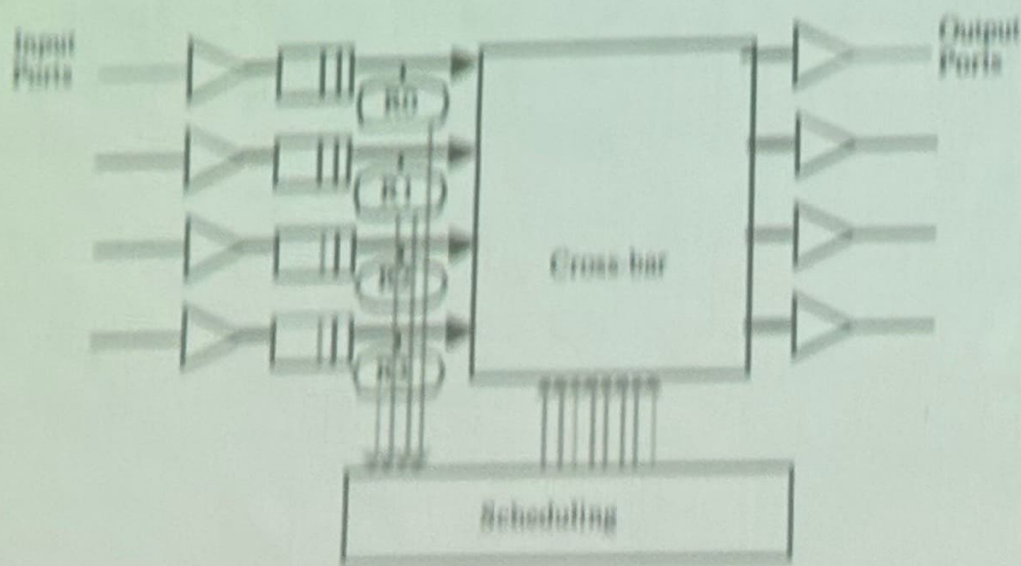
- Suffers from **head of line blocking**

- If head flit cannot move due to contention, another worm cannot proceed though links may be idle

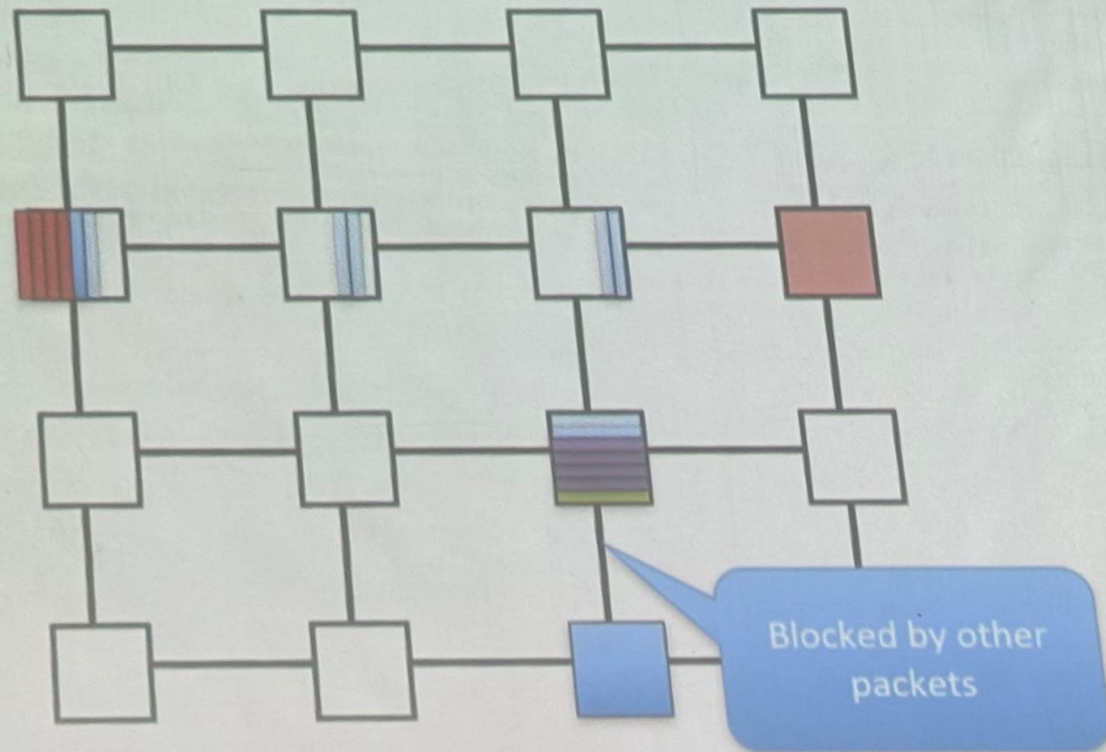


Head of Line Blocking

- A worm can be before another in the router input buffer
- Due to FIFO nature, the second worm cannot be scheduled even though it may need to access another output port

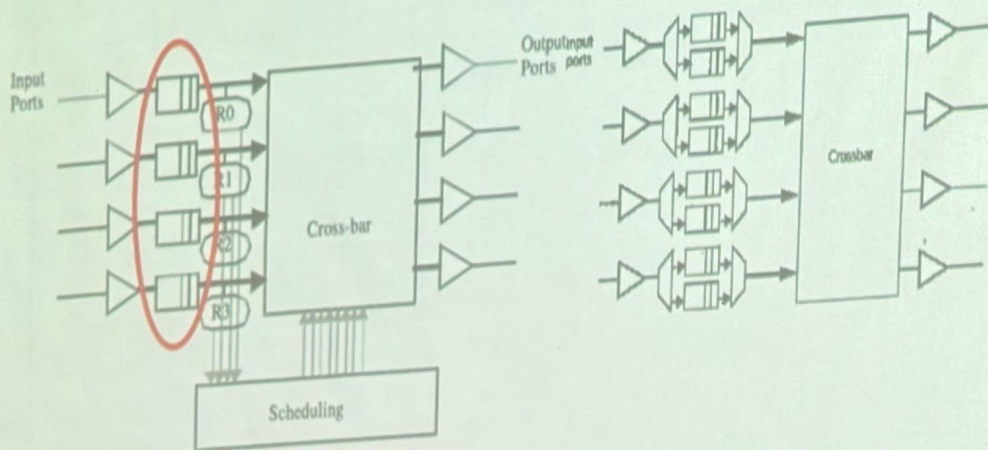


Head of Line Blocking



Virtual Channel Flow Control

- Idea: Multiplex multiple channels over one physical channel
- Divide up the input buffer into multiple buffers sharing a single physical channel
- Dally, "Virtual Channel Flow Control," ISCA 1990.



Virtual Channel Flow Control

- Idea: Multiplex multiple channels over one physical channel
- Divide up the input buffer into multiple buffers sharing a single physical channel
- Dally, "Virtual Channel Flow Control," ISCA 1990.

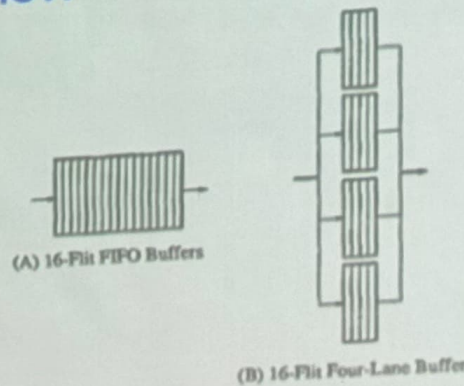


Figure 5: (A) Conventional nodes organize their buffers into FIFO queues restricting routing. (B) A network using virtual-channel flow control organizes its buffers into several independent lanes.

Other Uses of Virtual Channels

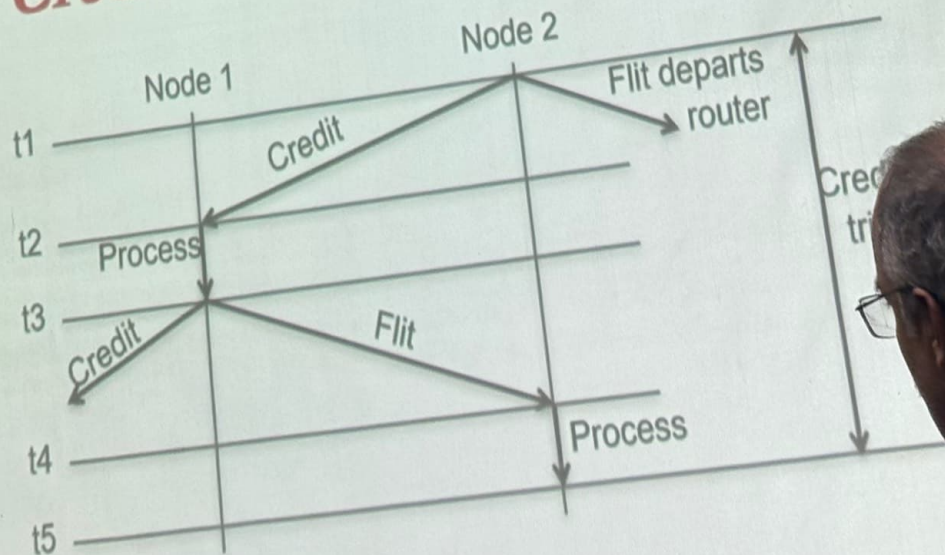
- **Deadlock avoidance**
 - Enforcing switching to a different set of virtual channels on some “turns” can break the cyclic dependency of resources
 - Enforce order on VCs
 - **Escape VCs:** Have at least one VC that uses deadlock-free routing. Ensure each flit has fair access to that VC.
 - **Protocol level deadlock:** Ensure address and data packets use different VCs → prevent cycles due to intermixing of different packet classes
- **Prioritization of traffic classes**
 - Some virtual channels can have higher priority than others

Communication

- Credit-based flow control
 - Upstream knows how many buffers are downstream
 - Downstream passes back credits to upstream
 - Significant upstream signaling (esp. for small flits)
- On/Off (XON/XOFF) flow control
 - Downstream has on/off signal to upstream
- ACK/NACK flow control
 - Upstream optimistically sends downstream
 - Buffer cannot be deallocated until ACK/NACK received
 - Inefficiently utilizes buffer space



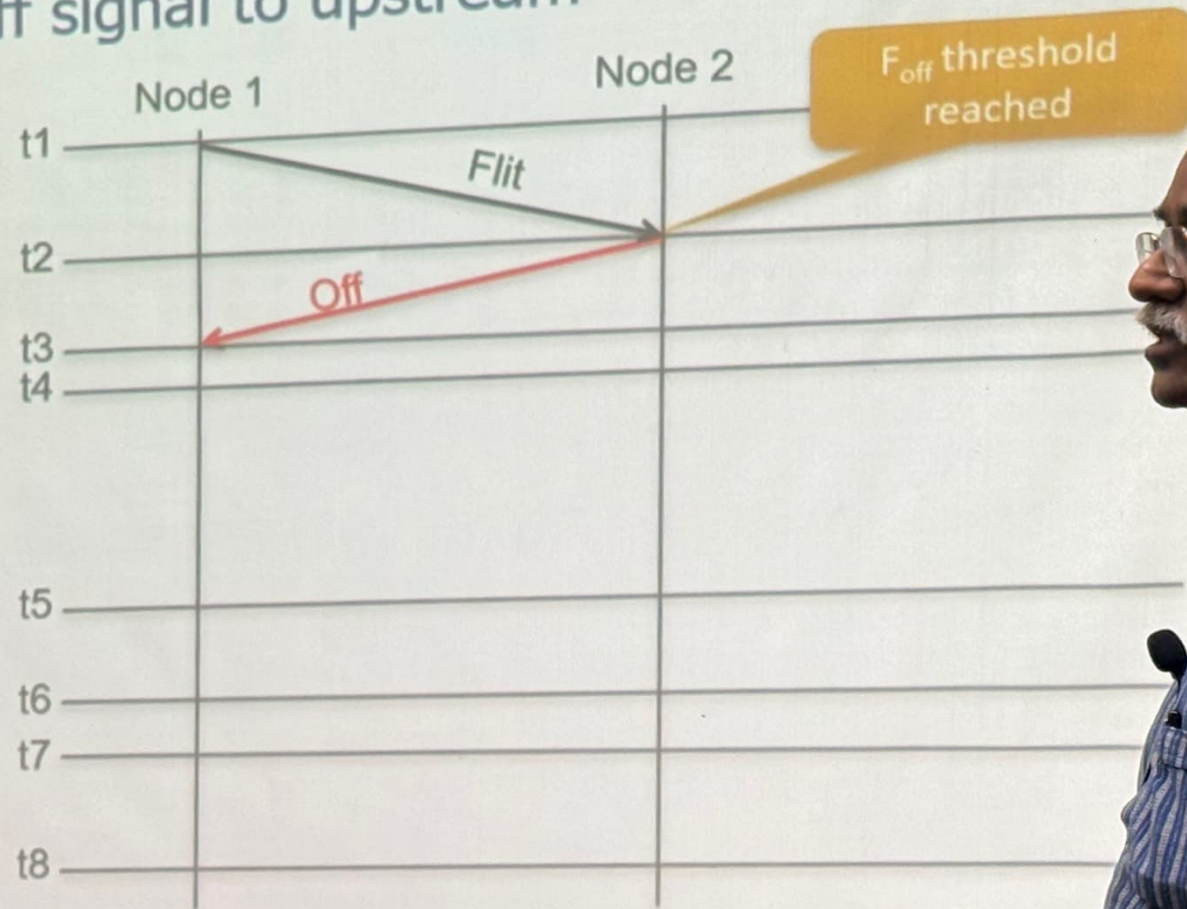
Credit-based Flow Control



- Round-trip credit delay:
 - Time between when buffer empties and when next flit can be placed in buffer entry
- Significant throughput degradation if there are few buffers
- Important to size buffers to tolerate credit turn-around

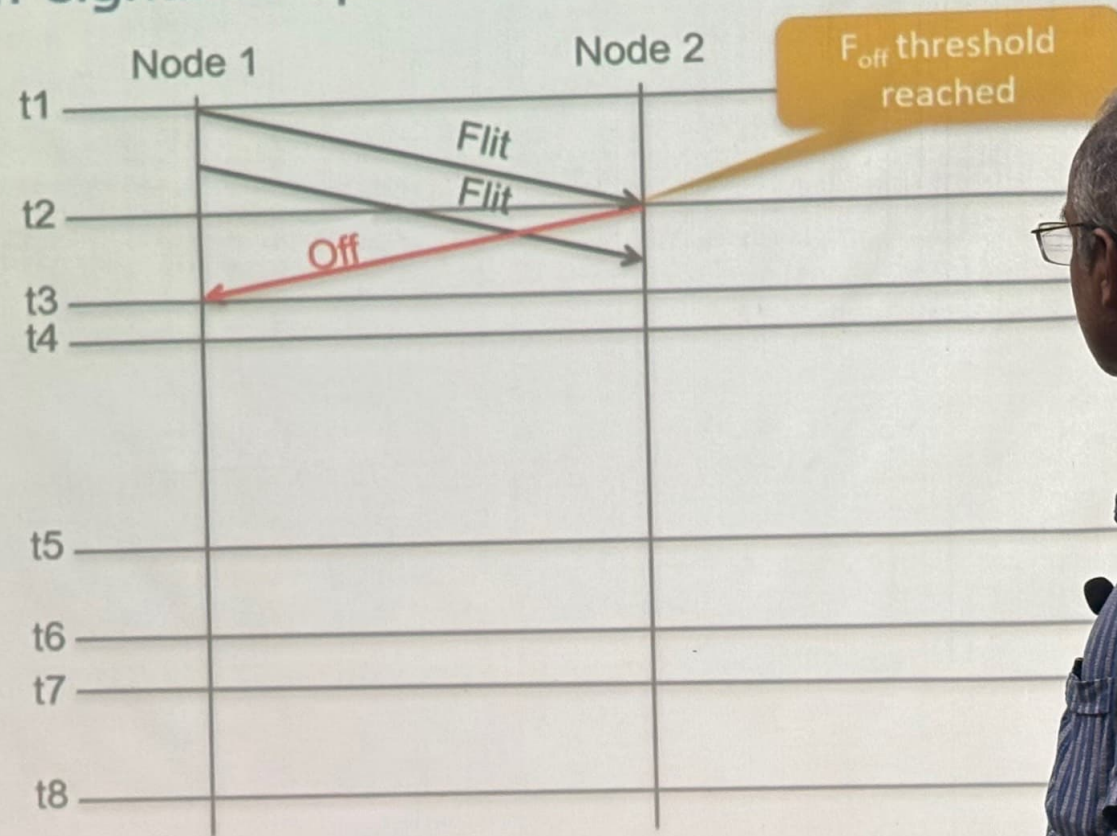
On/Off (XON/XOFF) Flow Control

- Downstream has on/off signal to upstream



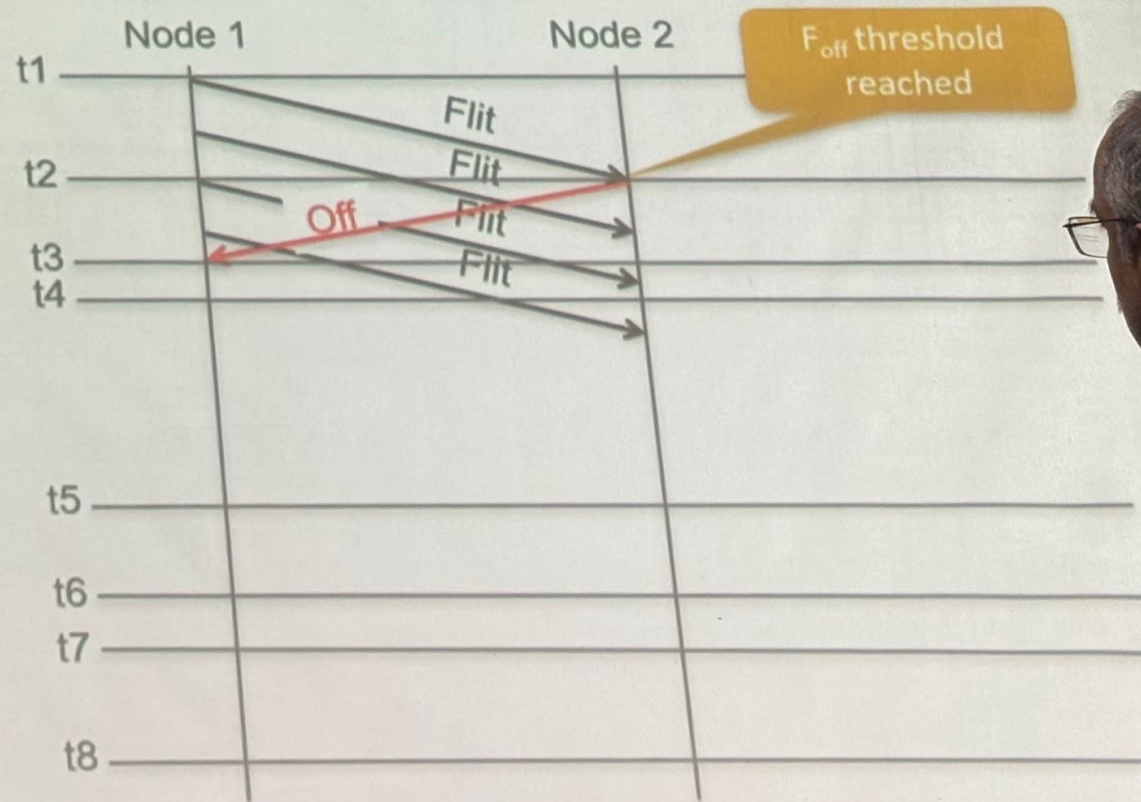
On/Off (XON/XOFF) Flow Control

- Downstream has on/off signal to upstream



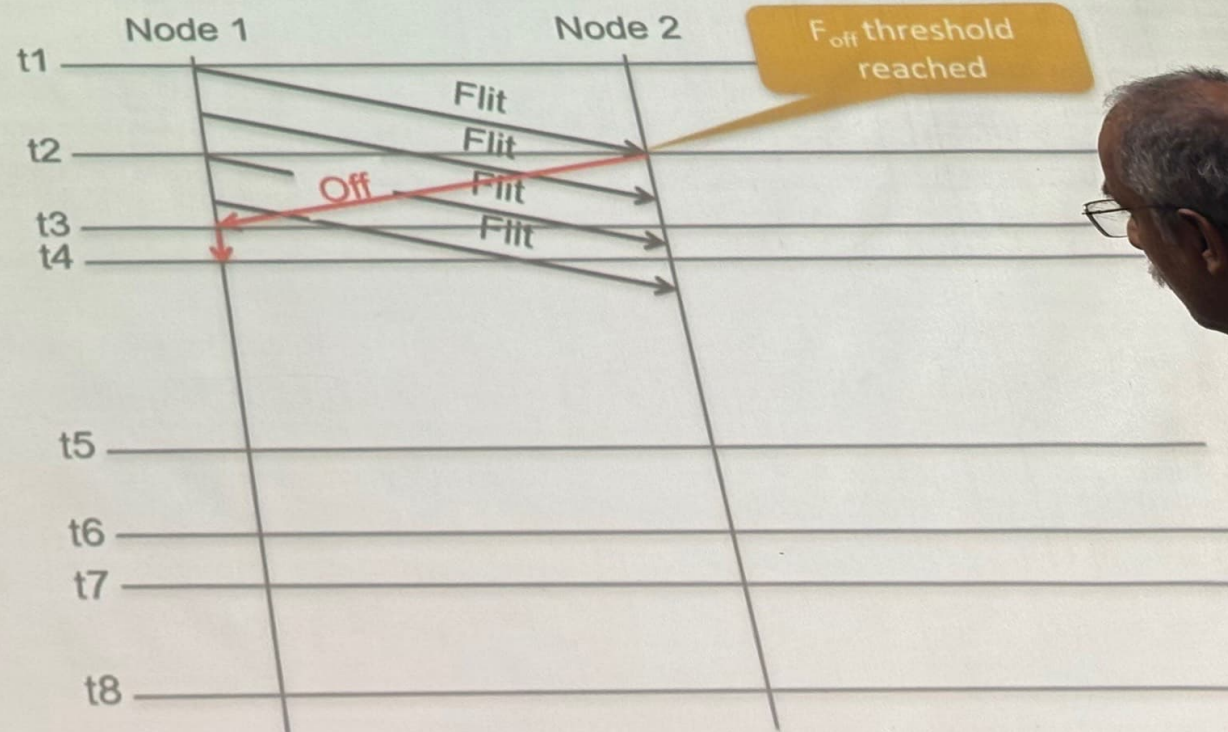
On/Off (XON/XOFF) Flow Control

- Downstream has on/off signal to upstream



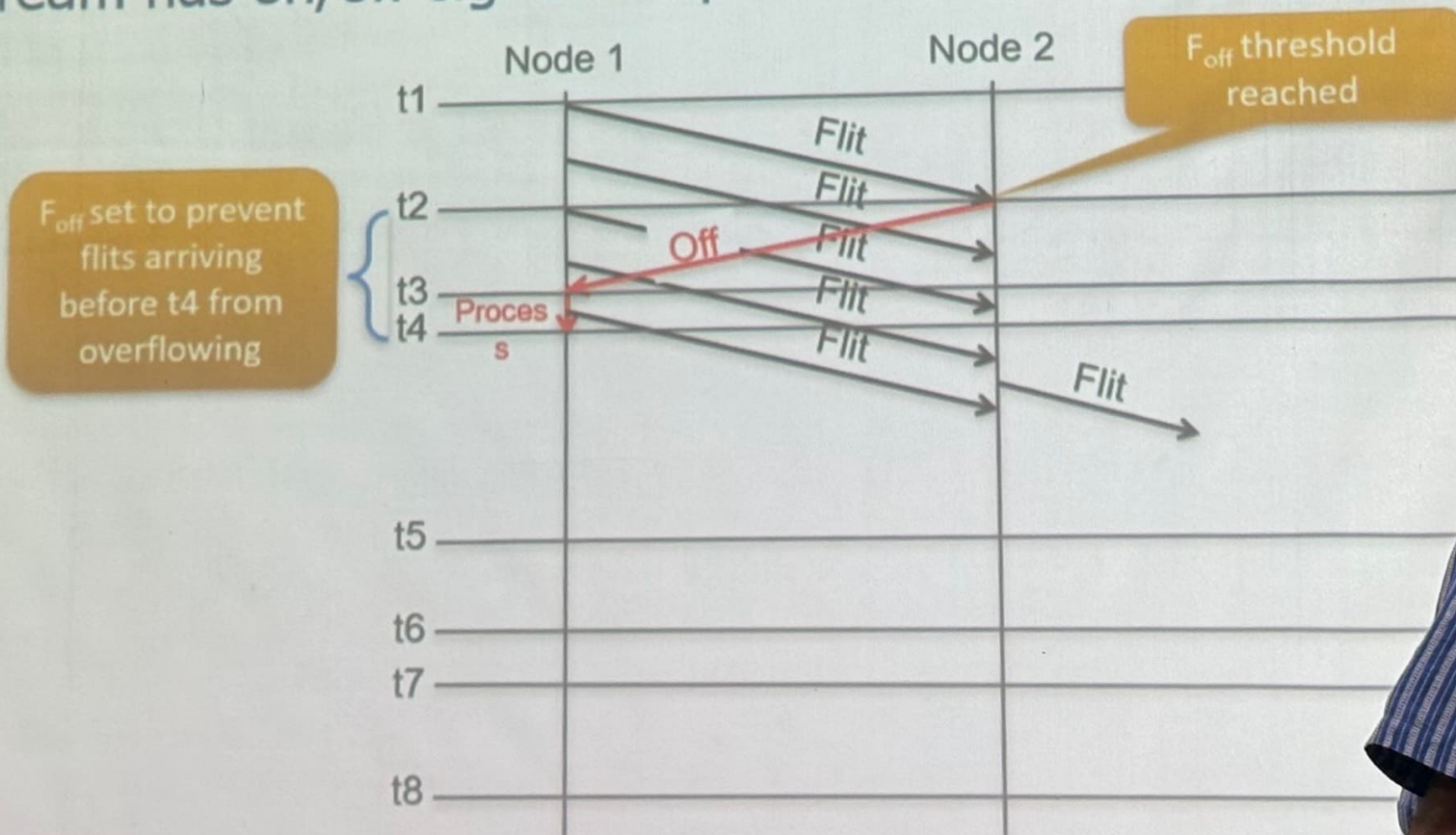
On/Off (XON/XOFF) Flow Control

- Downstream has on/off signal to upstream



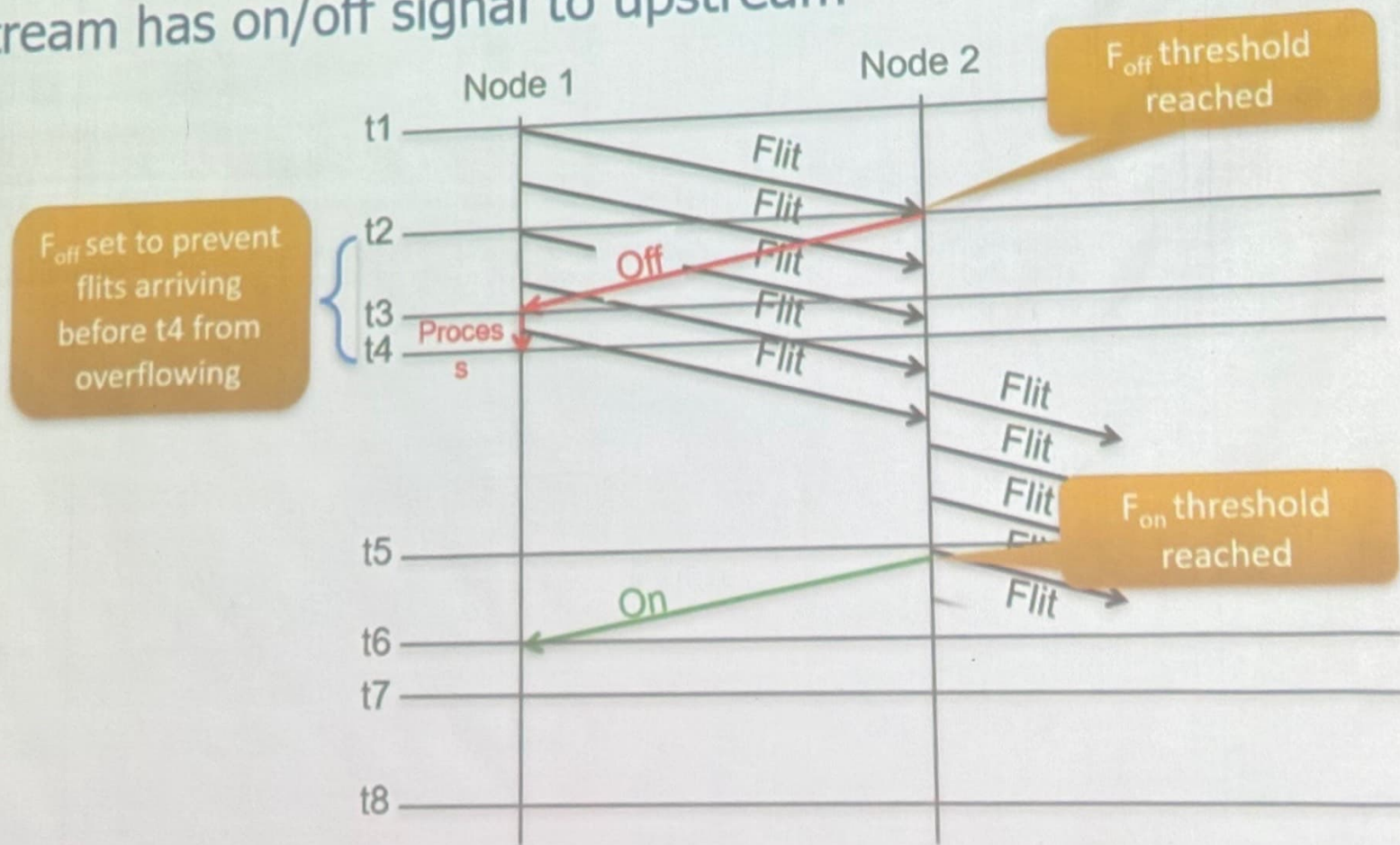
On/Off (XON/XOFF) Flow Control

- Downstream has on/off signal to upstream



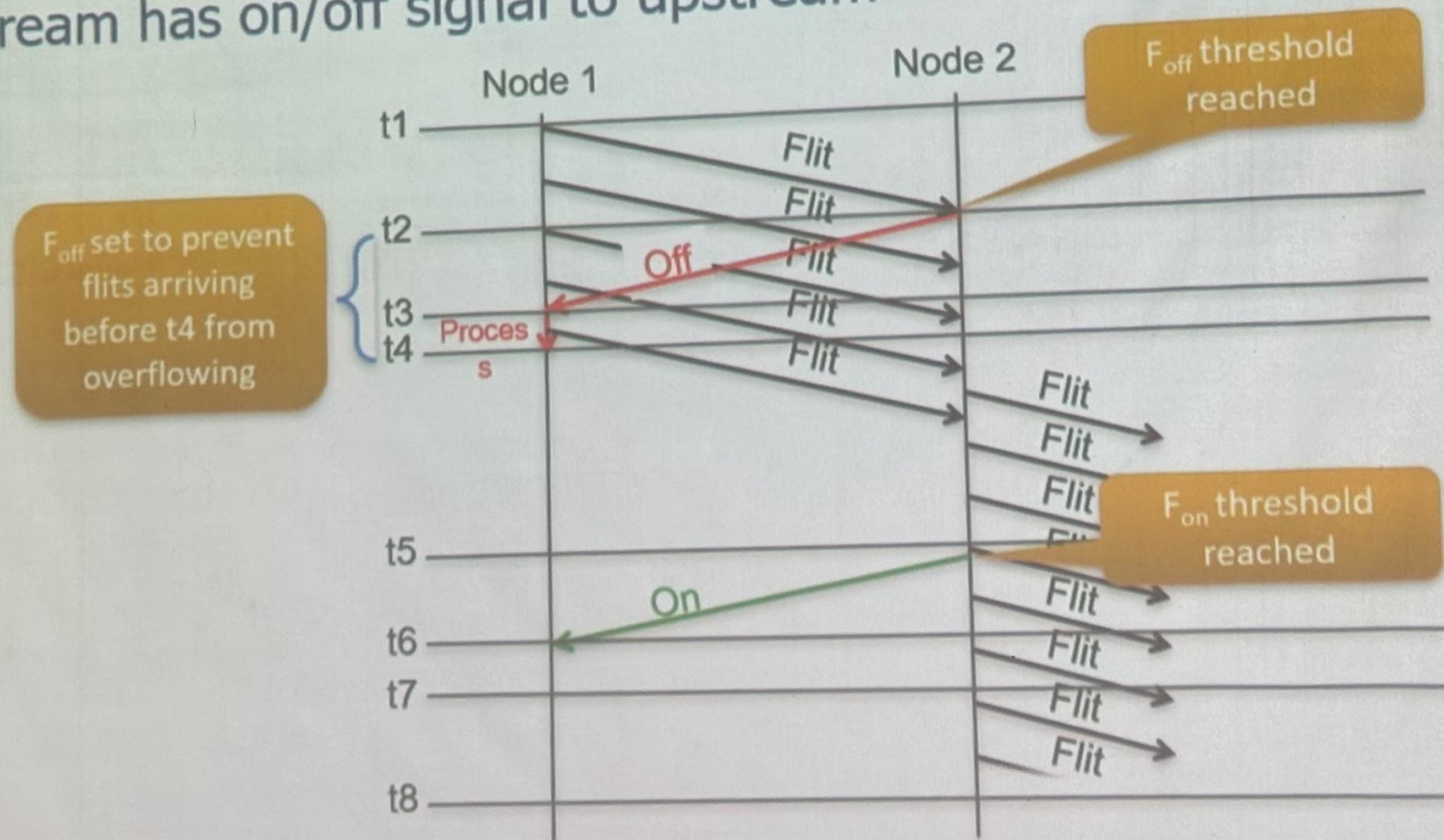
On/Off (XON/XOFF) Flow Control

- Downstream has on/off signal to upstream



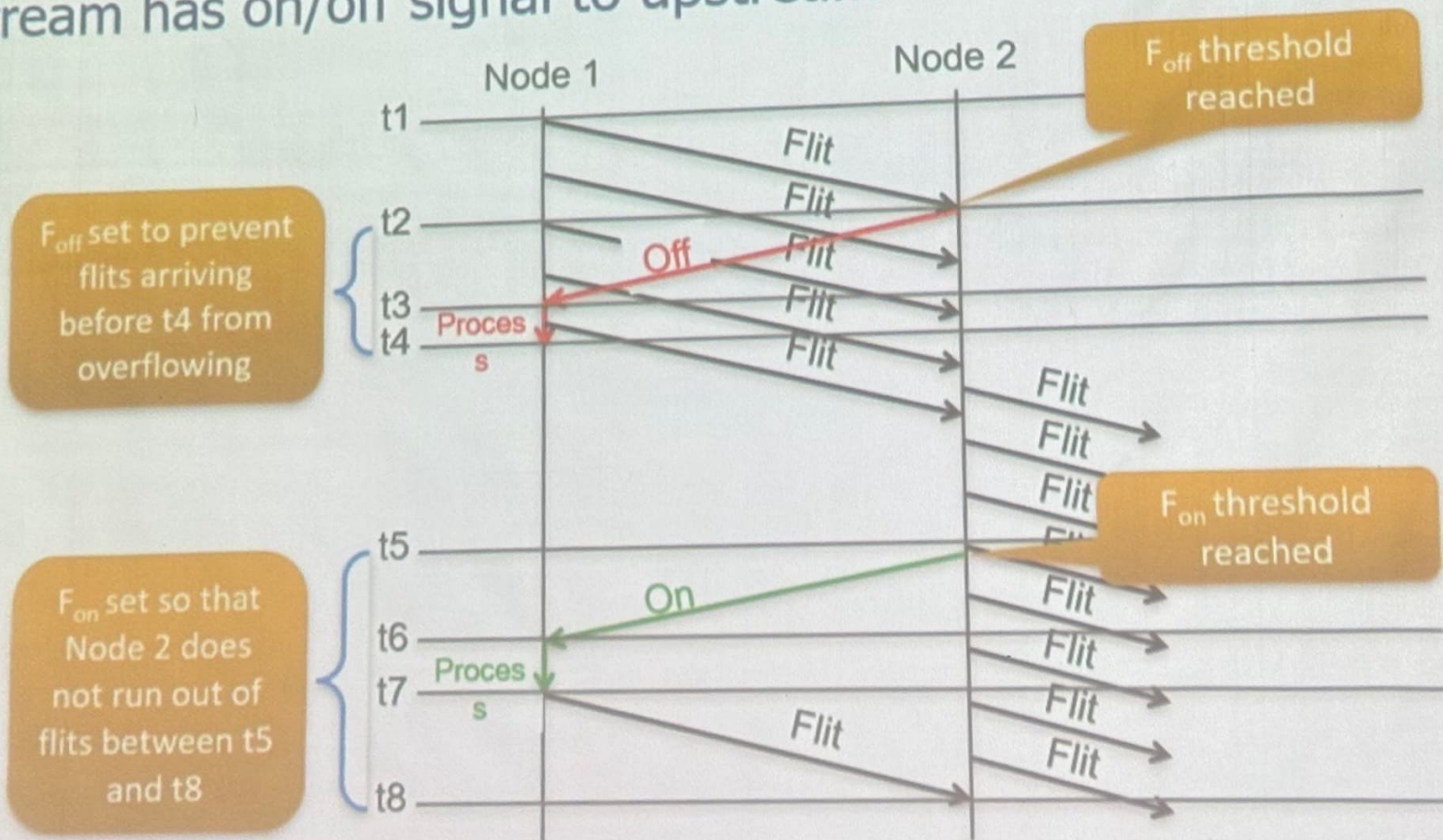
On/Off (XON/XOFF) Flow Control

- Downstream has on/off signal to upstream

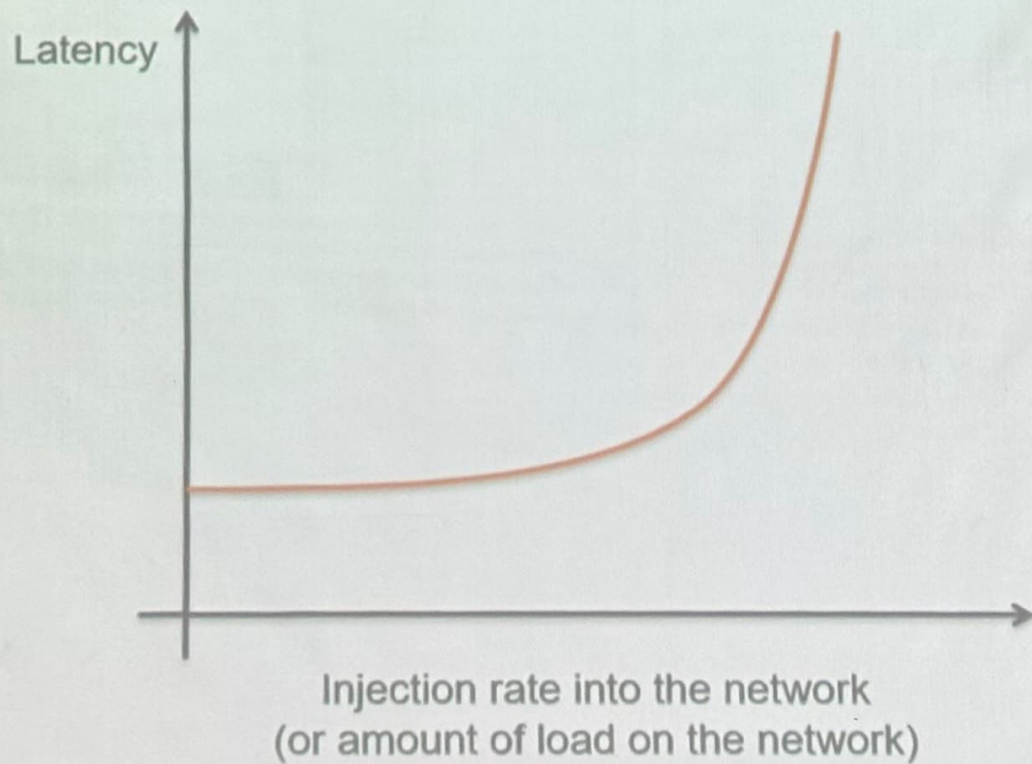


On/Off (XON/XOFF) Flow Control

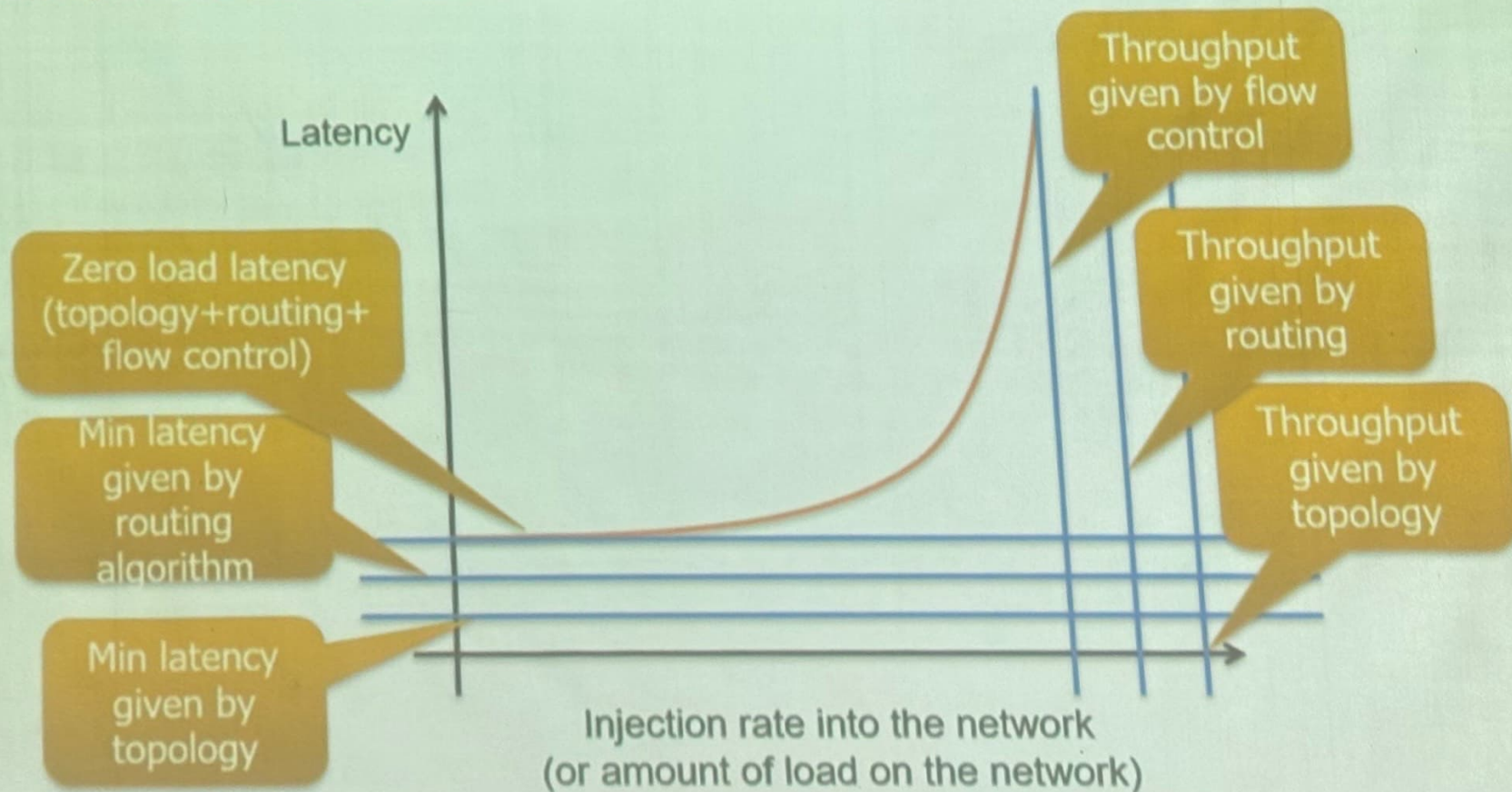
- Downstream has on/off signal to upstream



Interconnection Network Performance



Interconnection Network Performance



Ideal Latency

- Ideal latency
 - Solely due to wire delay between source and destination

$$T_{ideal} = \frac{D}{v} + \frac{L}{b}$$

- D = Manhattan distance
 - The distance between two points measured along axes at right angles.
- v = propagation velocity
- L = packet size
- b = channel bandwidth



Actual Latency

- Dedicated wiring impractical
- Long wires segmented with insertion of routers

$$T_{actual} = \frac{D}{v} + \frac{L}{b} + H \cdot T_{router} + T_c$$

- D = Manhattan distance
- v = propagation velocity
- L = packet size
- b = channel bandwidth
- H = hops
- T_{router} = router latency
- T_c = latency due to contention

Load-Latency Curve Examples

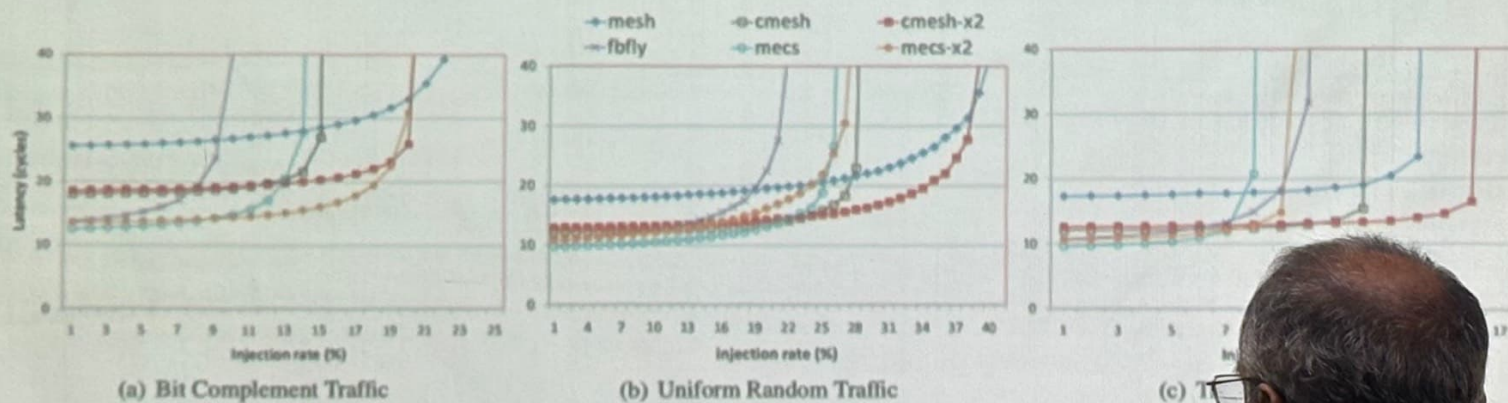


Figure 4. Load-latency graphs for 64-node mesh, CMesh, flattened butterfly and ME

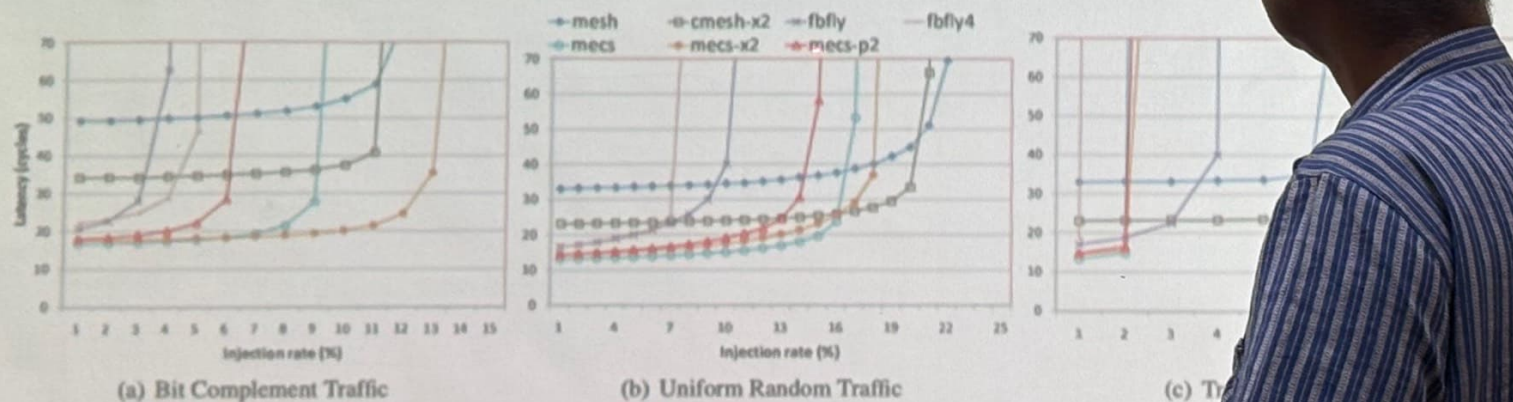


Figure 5. Load-latency graphs for 256-node mesh, CMesh, flattened butterfly and ME

Network Performance Metrics

- Packet latency (avg/max)
- Round trip latency (avg/max)
- Saturation throughput
- Application-level performance: execution time
- System performance: job throughput
 - Affected by interference among threads/applications

